Part IV

Application Engineering

Part IV: Overview

The main goal of application engineering is to derive a software product line application by reusing as many domain artefacts as possible. This is achieved by exploiting the commonality and the variability of the product line established in domain engineering. In this part you will learn how the orthogonal variability model is used in the application engineering sub-processes highlighted in Fig. IV-1 to:

- *Consider the commonality and the variability of the product line when defining the requirements for a specific application.*
- *Document the selected variants.*
- *Bind the selected variants from requirements to the architecture, to the components, and to the test cases.*
- *Estimate the impacts originating from differences between application requirements and domain requirements on architecture, components, and tests.*

The orthogonal variability model supports the consistent reuse of the domain assets, i.e. the domain requirements, architecture, components, and test cases.

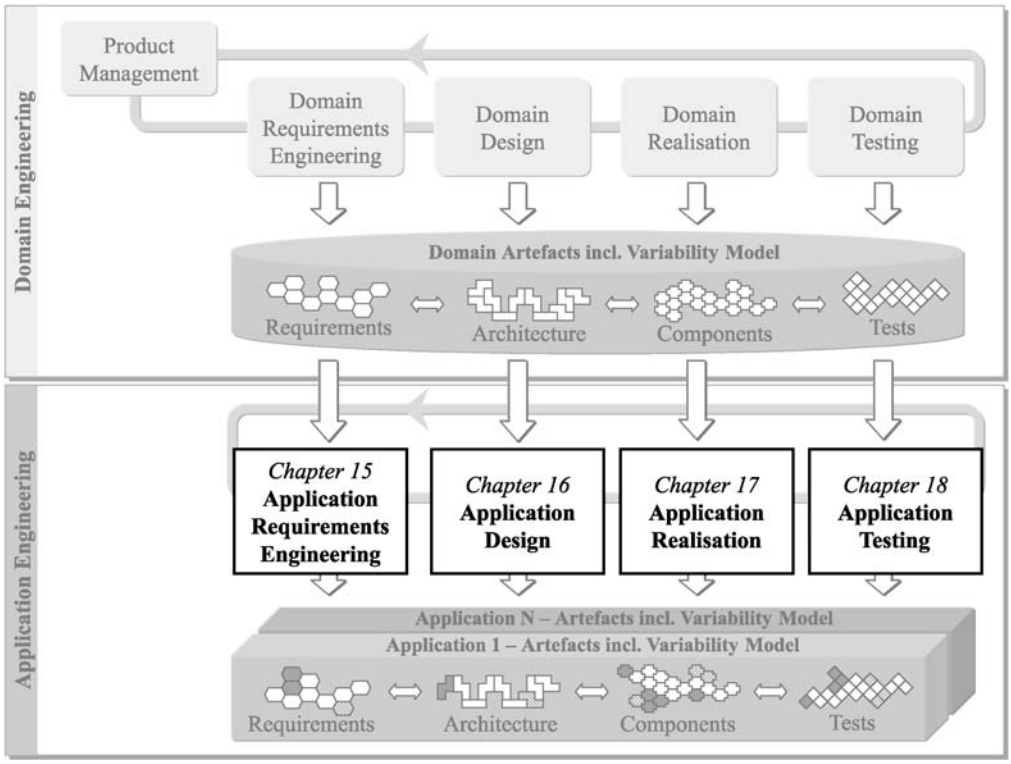


Fig. IV-1: Chapter overview of Part IV

Günter Halmans
Klaus Pohl

15

Application Requirements Engineering

In this chapter you will learn:

- *About the interrelations of the application requirements engineering sub-process with the product management, domain requirements engineering, and application design sub-processes.*
- *How to communicate the external variability and the commonalities of the product line to the stakeholders.*
- *How to identify deltas between stakeholder requirements and product line requirements.*
- *How to analyse and document changes such as adding new features or adapting product line features for a particular product line application.*

15.1 Introduction

Goal of application requirements engineering

The goal of application requirements engineering is to elicit and to document the requirements artefacts for a particular application and at the same time reuse, as much as possible, the domain requirements artefacts. The reuse of domain requirements artefacts for each application supports the overall goal of obtaining a high degree of domain artefact reuse.

Related sub-processes

The sub-processes and artefacts closely related to the application requirements engineering sub-process are highlighted in Fig. 15-1. Application requirements engineering is related to product management, domain requirements engineering, and application design. Product management defines the major features of the applications to be developed. Domain requirements engineering creates the domain requirements artefacts, which are reused for the application under consideration. The application requirements engineering sub-process reuses the domain requirements artefacts to define the application requirements artefacts. The application requirements artefacts serve as a basis for application design.

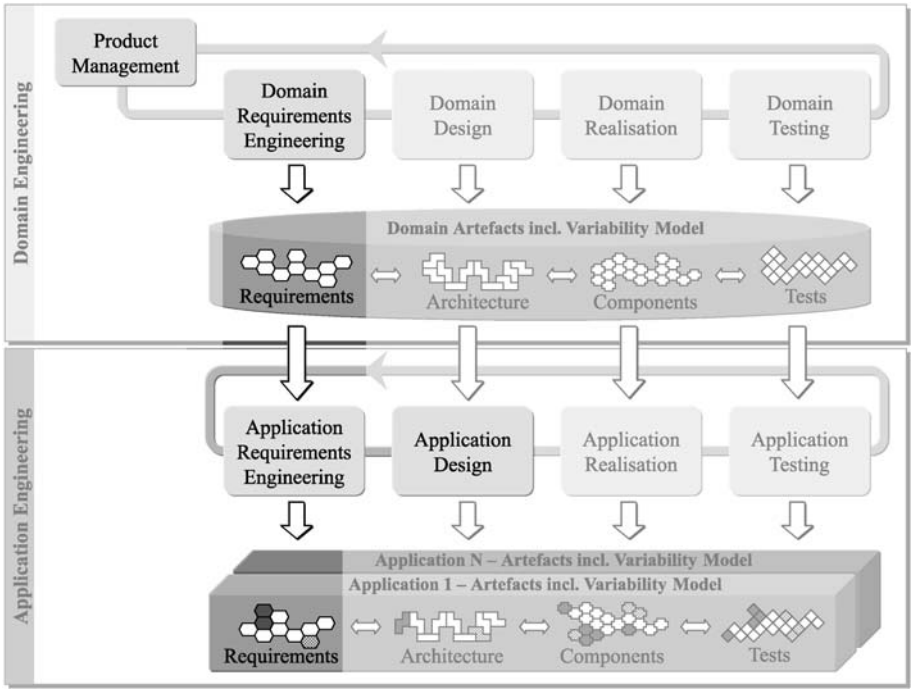


Fig. 15-1: Sub-processes and artefacts related to application requirements engineering

Communication with stakeholders

An essential activity of application requirements engineering is the communication of the domain requirements artefacts to the stakeholders. Hence, product managers and customers are typically involved in the application requirements engineering process.

Product managers and customers

Product managers determine the major features of the applications in the product portfolio based on their market and product (line) strategy. Customers demand an application that satisfies their individual needs at an affordable price. In cases where customers are known to the organisation, application requirements engineering communicates the commonality and external variability of the software product line to them. In the more common case where the customers are not personally known, product managers and marketing experts represent the customer as stakeholder of the application requirements engineering sub-process.[40] The communication about domain requirements artefacts[41] enables customers, or their representatives, to evaluate the extent to which the software product line can satisfy their needs.

Two application categories

Software product line applications can be divided into two basic categories with respect to the degree of domain artefact reuse. The first category comprises applications which have only requirements artefacts that are a subset of the domain requirements artefacts. The second category comprises applications which have requirements artefacts that are not part of the domain requirements artefacts. For applications of the first category, the domain requirements artefacts are communicated to the stakeholders, and the appropriate requirements are selected and documented. The second category asks for more effort for application engineering. As the applications of the second category cannot be realised by reusing domain requirements artefacts exclusively, the differences or *deltas* between domain requirements artefacts and application requirements artefacts have to be detected and documented. Requirements deltas lead to adaptation effort in all application engineering sub-processes and thus increase the price of the application.

Figure 15-2 shows the interrelations between application engineering and its related sub-processes. In the following, we describe each interrelation in detail.

15.1.1 Interrelation with Product Management

Application features

Product management defines the major application features (① in Fig. 15-2) for all applications of the product line. The development of the applications

[40] A distinction can also be made between the derivation of individual applications and the derivation of mass-market applications [Halmans and Pohl 2002]. Here, we distinguish between customers and product managers as stakeholders of the application requirements engineering process, which largely correlates with the differentiation between individual applications and mass-market applications.

[41] In the following, we use the term requirements as a synonym for requirements artefacts (Definition 5-2).

is supported by the commonality and variability of the platform. Application requirements engineering reuses the common parts and chooses the variant parts that are suitable to match the features defined by product management for the application. Certain features are application specific, i.e. they only apply for a single application. As the corresponding application requirements artefacts do not yet exist, application requirements engineering has to define them.

Feedback to product management

Application requirements engineering leads to new insights about required features, e.g. by communicating with different stakeholders. Based on the new insights application requirements engineering makes suggestions for additional and altered features that might be incorporated in the software product line (② in Fig. 15-2).

15.1.2 Interrelation with Domain Requirements Engineering

Domain requirements artefacts and variability model

Domain requirements engineering provides application requirements engineering with common and variable requirements artefacts and the domain variability model (③ in Fig. 15-2). Application requirements engineering employs the variability model to determine the variants as well as the corresponding domain requirements artefacts that can be reused for the application.

Additional needs

Application requirements engineering passes on requests for additional and altered domain requirements artefacts to domain requirements engineering (first bullet of ④ in Fig. 15-2). The requests typically originate from insights and experiences gained in assembling a specific application. In addition, customer requirements should be evaluated if they also affect other product line applications (i.e. if they rather represent needs of the domain than of a single application). If so, the requirements are passed on to domain requirements engineering to be elaborated further.

Application-specific requirements

Product management designates application-specific features to be worked out during application requirements engineering, e.g. if a lead product strategy is followed. If application-specific requirements address actual needs of the domain, they might be integrated into the domain artefacts. For this purpose, the application requirements artefacts are passed on to domain requirements engineering (second bullet of ④ in Fig. 15-2). Before these artefacts can be integrated into the product line, a decision within the domain engineering process has to be made and, if the decision is to integrate them, the domain artefacts have to be adapted to incorporate the new requirements.

15.1.3 Interrelation with Application Design

Application requirements specification

The main output of application requirements engineering is the application requirements specification (⑤ in Fig. 15-2) which is a complete specification of the application. It includes the application variability model, which is derived from the domain variability model, the requirements artefacts that are reused from the domain artefacts, and the requirements deltas. Requirements deltas are determined by analysing the requirements posed by the customer or product manager and comparing them with domain requirements artefacts. The application requirements specification is described in more detail in Section 15.5. Based on the application requirements specification (and the reference architecture), application design derives the application architecture.

Effort for deltas

Application requirements engineering typically involves trade-off decisions with regard to the requirements posed by a customer or representative. The realisation effort for the requirements depends on the degree of reuse that can be achieved. Requirements deltas, such as performance requirements that are tighter than anticipated by the product line, may involve significant modifications of the architecture and the reusable components. As such modifications affect the development costs for the application, trade-off decisions are necessary on whether to accept a higher price or to abstain from the specific requirement that causes cost-intensive modifications. Application design has to support such decisions by providing an effort evaluation of requirements deltas (⑥ in Fig. 15-2).

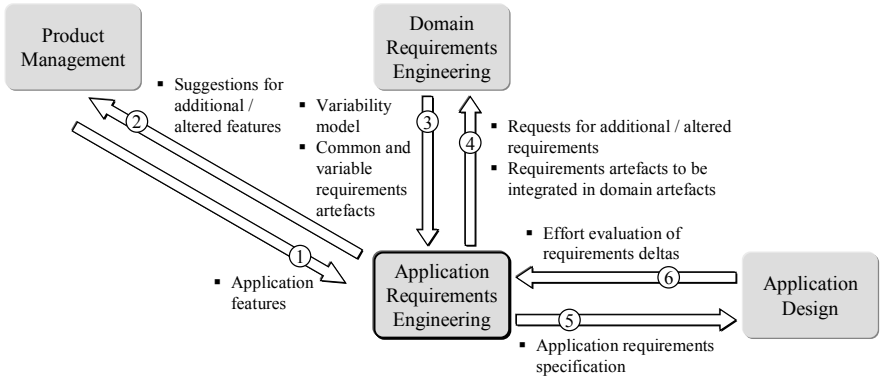


Fig. 15-2: Information flows between application requirements engineering and other subprocesses

15.2 Application Requirements Engineering Activities

Stakeholder requirements

In the following, we do not distinguish whether the need to develop application-specific requirements artefacts originates from product management, from customer needs, or from any other source. We treat the different cases similarly by the assumption that there are some stakeholders who pose requirements with respect to the considered application. We define these requirements as *stakeholder requirements*:

> ***Definition 15-1:*** *Stakeholder Requirements*
>
> Stakeholder requirements are requirements that stakeholders state for a particular application, i.e. requirements that the stakeholders expect to be fulfilled by the application.

Application requirements

Due to trade-off decisions made in the application requirements engineering process, the initial stakeholder requirements are not necessarily identical with the resulting *application requirements*. We define application requirements as follows:

> ***Definition 15-2:*** *Application Requirements*
>
> Application requirements are requirements that completely specify a particular product line application.

Decision options

The agreement about the application requirements is a result of the application requirements engineering process. The following options exist with regard to a particular stakeholder requirement:

- The stakeholder requirement can be completely fulfilled by an application requirement or set of application requirements.
- The stakeholder requirement can be partially fulfilled by an application requirement or set of application requirements.
- The stakeholder requirement cannot be fulfilled by any application requirement or set of application requirements.

The decisions about stakeholder requirements affect the interrelations between stakeholder requirements, application requirements artefacts, and domain requirements artefacts:

- *Interrelation between stakeholder requirements and application requirements artefacts*

Feasibility

Normally, stakeholder requirements should be fulfilled by the application requirements artefacts. Yet, if the realisation of requirements deltas

leads to a significant effort, the stakeholders may decide that their requirement should only be partially fulfilled by the application or not fulfilled at all. We address this topic in Section 15.4.4.

- *Interrelation between stakeholder requirements and domain requirements artefacts*
  The requirements engineer maps stakeholder requirements to domain requirements artefacts with the goal to find domain requirements artefacts that satisfy the particular stakeholder requirement. In case a specific domain requirements artefact satisfies a particular stakeholder requirement, the domain requirements artefact can be reused. In case a particular stakeholder requirement cannot be fulfilled by domain artefacts, adaptation effort is necessary to satisfy the stakeholder requirement. Then, a trade-off decision is necessary on whether the application requirements artefacts must fully comply with the stakeholder requirement or may be adapted to eliminate the delta (or at least reduce the adaptation effort). In the latter case, the stakeholder requirement is not fulfilled (completely).

Reuse

- *Interrelation between application and domain requirements artefacts*
  An application requirements artefact is identical to a domain requirements artefact if the domain requirements artefact satisfies a particular stakeholder requirement and thus can be completely reused. An application requirements artefact has a delta to a particular domain requirements artefact in case the stakeholder requirement cannot be completely satisfied by a domain requirements artefact.

Requirements deltas

For the elicitation and documentation of application requirements, the following three activities are essential:

- *Communicating the commonality and external variability of the product line*[42]
  The goal of this activity is to make the stakeholder aware of the capabilities of the product line and to elicit application requirements. By considering the commonality and variability of the product line in application requirements engineering, the level of domain artefact reuse can be increased [Halmans and Pohl 2001]. The orthogonal variability model plays a central role in this activity as it enables the requirements engineer to communicate the relevant variation points, variants, and their dependencies to the stakeholder (① in Fig. 15-3). Additionally, the variability model and its traceability links to domain requirements artefacts enable the requirements engineer to describe the functionality and quality of a particular variant. The stakeholders survey the product line

Awareness of product line capabilities

[42] Depending on the knowledge the stakeholder already has about the commonality and variability of the product, the steps of this activity are more or less distinct.

commonality and the external variability and communicate their requirements for the application to the requirements engineer (② in Fig. 15-3). The requirements engineer collects the domain requirements artefacts to be reused for the application. The result of the activity is a set of domain requirements artefacts, which may not completely fulfil the stakeholder requirements.

Estimation of realisation effort

- *Evaluating deltas between domain and application requirements*
  Deltas between domain and application requirements occur when stakeholder requirements cannot be completely satisfied by domain requirements artefacts. These deltas have to be evaluated with respect to the required realisation effort. During the evaluation process, first, deltas to the domain variability model caused by the stakeholder requirements are analysed. Second, the impact of the variability model deltas on the corresponding domain requirements artefacts is analysed. The results of this analysis are variability model and requirements artefact deltas. They are communicated to the application architect who estimates the realisation effort based on the deltas. The feedback on the estimated realisation effort (③ in Fig. 15-3) allows the stakeholder to decide whether the requirements artefact deltas should be realised or not. The stakeholder communicates the decision to the requirements engineer (④ in Fig. 15-3). As a result of the delta evaluation activity, the application requirements, and the corresponding requirements artefact and variability model deltas, are defined.

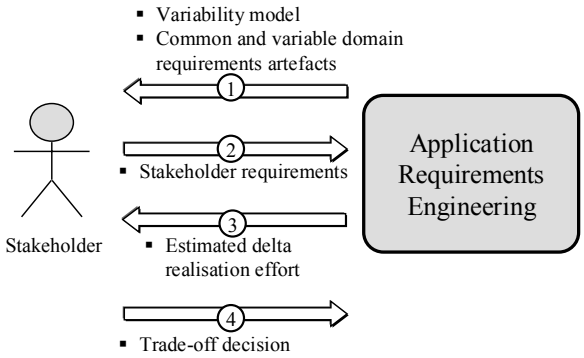


Fig. 15-3: Information flows with respect to the stakeholder

Basis for later development phases

- *Documentation of application requirements*
  The two activities described above result in a documentation that includes the application requirements artefacts, the deltas between application and domain requirements artefacts, and the traces between application requirements artefacts and the corresponding domain

requirements artefacts. In addition, the application variability model (which is the result of incorporating the variability model deltas) and the traceability links of application requirements to the application-specific variability model are documented. Moreover, the estimated realisation costs are related to the deltas to keep decisions about the deltas traceable. The resulting application requirements specification is the basis for the later development phases.

15.3 Communication of the Product Line Variability

Requirements artefacts and variability

This section focuses on the communication of external variability, using the orthogonal variability model, to the stakeholders. The variability model provides a coherent view of the variability of the product line. The requirements engineer navigates between the variability model and the different requirements artefacts to supply stakeholders with more detailed information, e.g. about the functionality and quality of the variants under consideration.

15.3.1 Variation Points and Variants

The communication of external variability based on the orthogonal variability model typically starts with the variation points that provide the topmost level of abstraction. Communicating a single variation point involves:

1. Communicating the variants related to the variation point as well as the variability dependencies and the alternative choices defined for the variation point.
2. Communicating dependent variation points and/or variants by following the existing constraint dependencies.

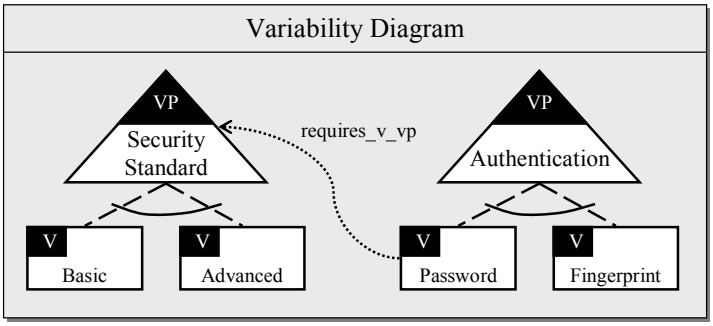


Fig. 15-4: Variation points ‘security standard’ and ‘authentication’

Example 15-1 illustrates the communication of variation points and variants to a stakeholder. For a more detailed description of communicating the variability of the software product line, see [Halmans and Pohl 2003].

> ***Example 15-1:*** *Communicating a Variation Point*
>
> Figure 15-4 depicts an extract of an orthogonal variability model. It shows the two variation points 'security standard' and 'authentication'. The stakeholder is interested in the variation point 'authentication'. The following information can be communicated to the stakeholder:
>
> - The product line provides two authentication mechanisms. Exactly one of these has to be selected: authentication via 'Password' or authentication via 'fingerprint'.
> - In case of the variant 'password', additionally, the variation point 'security standard' has to be bound. The stakeholder has to select either 'basic' or 'advanced' security.

15.3.2 Domain Requirements Artefacts

Different views

To make a decision for or against a variant the stakeholder may need more detailed information concerning the functionality or quality associated with the variant. For instance, the features related to the variant under consideration are used to provide a management view. The related model-based requirements such as a class diagram provide a more detailed, solution-oriented view. Example 15-2 illustrates the communication of the domain requirements artefacts that are related to the variant 'password'.

Navigating the variability model

Having considered the domain requirements artefacts of a certain variant, different strategies can be followed to find the next variation point and variants to be considered. Two basic options are:

- The requirements engineer can communicate the next variation point at the topmost level of abstraction, which has not yet been considered. Thus the variation points that may affect coarse-grained properties of the resulting application are bound first.
- If a certain domain requirements artefact is associated to more than one variant, the requirements engineer can communicate all associated variants and thereby bind the variability related to the considered artefact before considering other artefacts.

Example 15-2: *Communicating the Details of a Variant*

The stakeholder is interested in more details about the variant 'password' depicted in Fig. 15-5 . By following the traceability link to the domain requirements artefacts, the requirements engineer finds the use case 'authentication by password'. By considering the use case description, the stakeholders get a more detailed idea of the benefits of this use case (the use case description is not depicted in Fig. 15-5).

The variant 'password' is interrelated with the variation point 'security standard'. Thus, to explain the considered variant 'password' in more detail, the requirements engineer shows the associated security requirements of the 'basic' variant and the 'advanced' variant to the stakeholder (see Fig. 15-5).

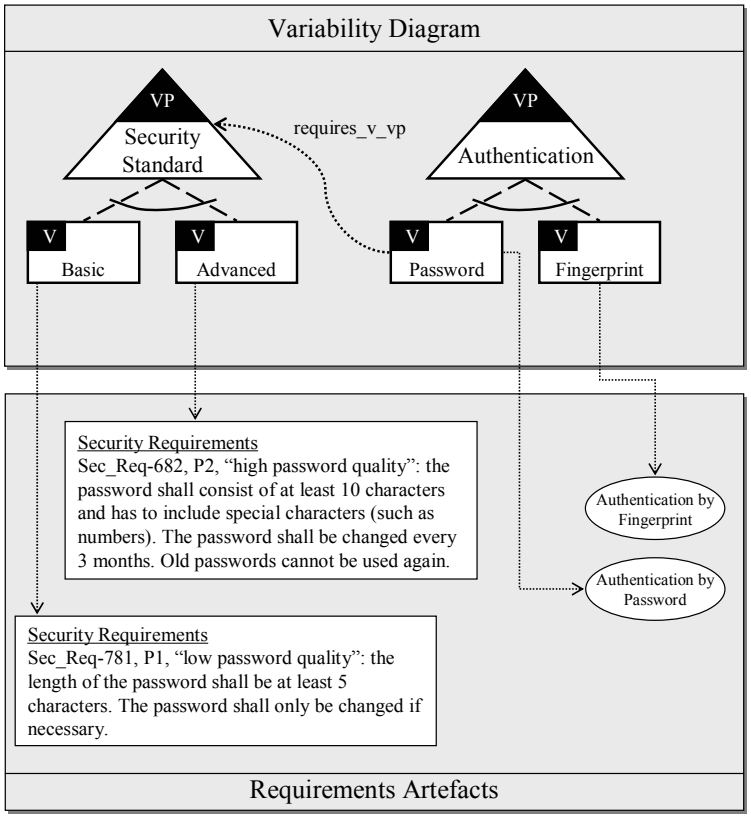


Fig. 15-5: Traceability links between variants and domain requirements artefacts

15.3.3 Result of the Communication Activity

The result of the communication activity is a classification of stakeholder requirements:

Reuse of domain artefacts

a) Stakeholder requirements that can be satisfied by binding variability defined in domain requirements artefacts.

   The domain requirements artefacts that satisfy these stakeholder requirements are documented as application requirements in a way that supports the reuse of domain artefacts in application engineering. We deal with this issue in Section 15.5.

Analysis of deltas

b) Stakeholder requirements that do not correspond to domain requirements artefacts.

   In this case, deltas between domain requirements artefacts and the application requirements artefacts that satisfy the stakeholder requirements exist. Section 15.4 deals with the analysis of these deltas.

15.4 Analysis of Requirements Deltas

Decision support

The main goal of delta analysis is to support the decision on whether the deltas should be realised for the application or not. We analyse the deltas caused by application requirements with respect to the variability model, domain requirements artefacts, and the application architecture. Based on the results of the analysis, the stakeholders decide whether the delta shall be realised in the application or not.

15.4.1 Variability Model Deltas

Variability model deltas are differences between the domain variability model (see Section 2.5) and the application variability model (Section 2.7). There are two types of such deltas:

- *Part of the existing external variability has to be modified*: A new variant must be added, or a variability or constraint dependency must be modified.
- *An invariant part must be turned into a variable part*: Part of the common requirements must be made variable. In this case, the external variability is extended by the introduction of a new variation point.

15.4.2 Impact on the Variability Model

Adding a new variant

In case the stakeholder requires, say, new functionality or quality with respect to a given variation point, a new variant must be included in the application variability model (Example 15-3). In case the stakeholder wishes

to change the functionality or quality of an existing variant, a new variant (representing the changed variant) representing the changed functionality or quality is defined. Thus, the two variants (the original one and the changed one) are selectable.

Modification of dependencies

The stakeholder may also demand modifications of the variability or constraint dependencies. For example, the following cases may occur:

- The stakeholder rejects a mandatory variant.
- The stakeholder selects a different number of variants than the range of an alternative choice permits.
- The stakeholder selects two variants that are related by an "exclude" dependency.
- The stakeholder selects only one out of two variants that are related by a "requires" dependency.

> ***Example 15-3:*** *Adding a New Variant*
>
> The stakeholder might demand an additional notification mechanism that enables the home automation system to notify the owner of the home about alarms via SMS (Short Message Service). This delta can be realised by adding the new variant 'SMS information' (Fig. 15-6).

Application as product line

Modifications of variability and constraint dependencies often occur when the stakeholder uses the application as a component of a larger system, which itself is offered in different variants. Then, the permissible combinations of variants have to be aligned with the variability of the larger system. Example 15-4 illustrates the removal of an alternative choice.

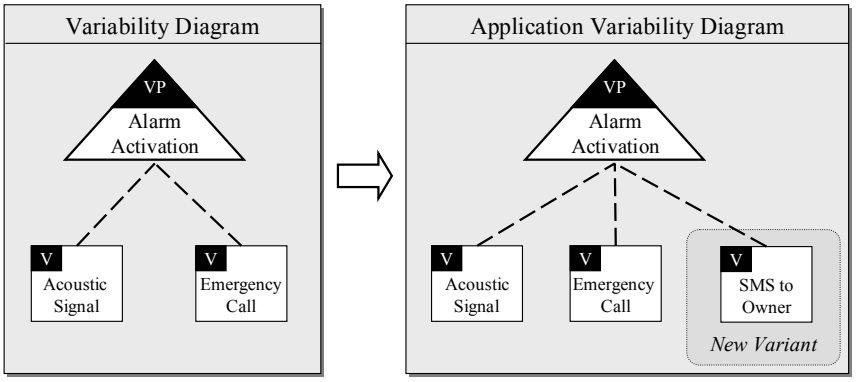


Fig. 15-6: Example of adding a new variant due to a requirements delta

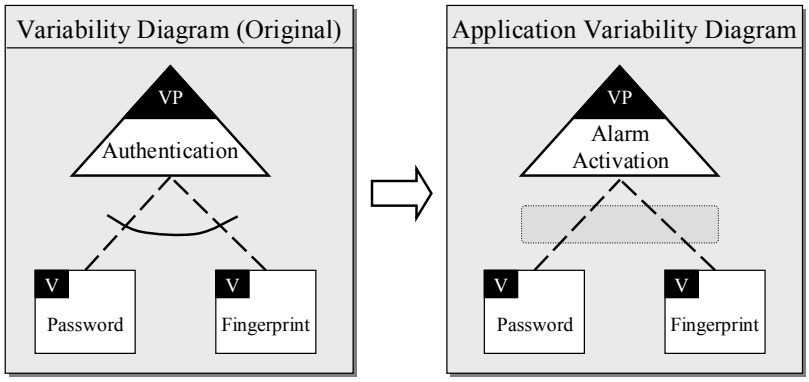


Fig. 15-7: Example of a requirements delta realised by removing an alternative choice

Example 15-4: *Removing an Alternative Choice*

Figure 15-7 shows an alternative choice with two variants 'password' and 'fingerprint'. The stakeholder requires an application that can be configured to support any combination of variants, including no authentication. In order to satisfy these requirements, the alternative choice is removed resulting in two independent optional variability dependencies. Note that the same result could be achieved by defining the range [0..2] for the alternative choice.

Adding a new variation point

Stakeholder requirements may also require that a commonality is defined as variability in the application variability model. Making common parts variable is reflected in the variability model by adding a new variation point together with the required variants.

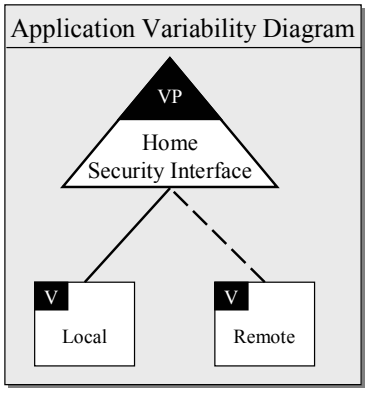


Fig. 15-8: Example of adding a new variation point due to a requirements delta

Example 15-5: *Introduction of a New Variation Point*

A home security product line provides a common mechanism to set up the home security system via a local user interface. The stakeholder needs a home security system that also offers remote user interfaces. Hence, the stakeholder requires a variation point 'home security interface' with two variants 'local' and 'remote'. Local access must be available in each home automation system while the remote variant is optional. The corresponding excerpt of the variability model is shown in Fig. 15-8.

15.4.3 Impact on Requirements Artefacts

For each change to the variability model, the application requirements artefacts that are affected by the change have to be determined. The application requirements artefacts as well as the artefact dependencies to the variants may need to be adapted. In the following, we analyse the required changes of application requirements artefacts with respect to the different kinds of changes in the variability model.

Changes in variation points and variants

The first type of requirements artefact deltas comprises changes caused by the introduction of new variants and/or new variation points. An example of such an adaptation is depicted in Fig. 15-5 and described in Example 15-6.

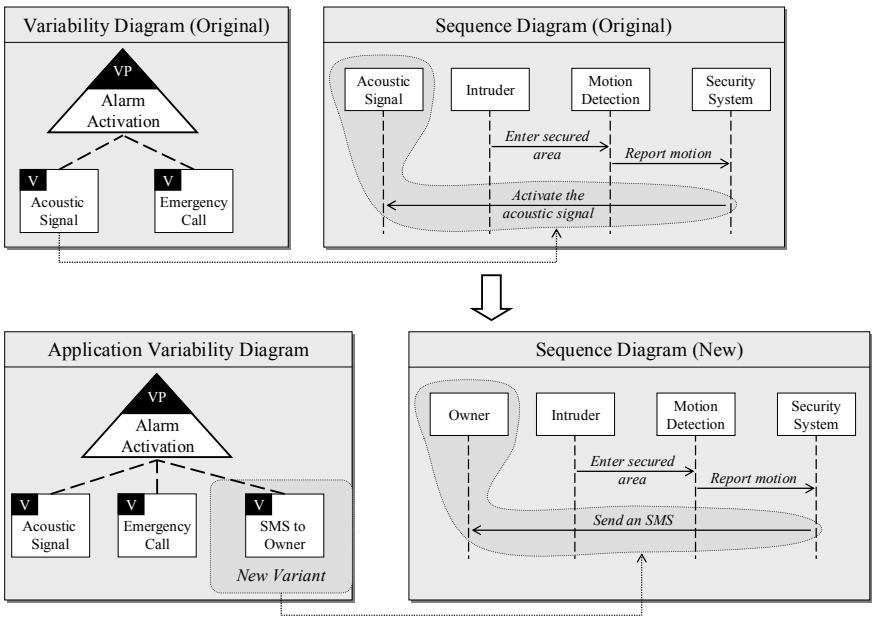


Fig. 15-9: Adaptation of a sequence diagram due to an adapted variant

> ***Example 15-6:*** *Introduction of a New Scenario*
>
> The upper half of Fig. 15-9 shows a variant ‘acoustic signal’ and its associated scenario. The scenario includes an actor ‘acoustic signal’ and an interaction that activates the acoustic signal. As the stakeholder requires an SMS notification, the variant ‘SMS to owner’ is introduced (lower half of Fig. 15-9). The variability model delta is incorporated into the new scenario by changing the scenario elements that deal with the acoustic signal in the original scenario.

Changes in dependencies

The second type of requirements artefact deltas comprises changes caused by adaptations to variability and constraint dependencies. Changes to variability dependencies may lead to restrictions on the permissible combinations of variants. Grouping a set of optional variability dependencies by an alternative choice with a range of [1..1] is an example of such a restriction. Some of the combinations of variants that were originally permitted become invalid. Requirements artefacts that describe such combinations have to be adapted. Changes to dependencies may also result in an extension of the possible combinations. This case is illustrated in Example 15-7.

> ***Example 15-7:*** *Impact of Variability Dependency Changes*
>
> In Example 15-4 the alternative choice for the two variants ‘password’ and ‘fingerprint’ is removed resulting in two independent optional variability dependencies. Consequently, requirements artefacts are needed that describe each combination of variants: no authentication, one kind of authentication, or both. For instance a scenario can be defined that describes unlocking a door using both password and fingerprint authentication together.

Impact on associated variants

Each requirements artefact may be associated with more than one variant. Hence, the requirements artefact delta to a particular application requirement may influence other associated variants. The requirements engineer stops the impact analysis when the influences of all changes in the variability model on the requirements artefacts have been analysed. For all changes, the stakeholders are involved to decide how the application requirements should be adapted.

15.4.4 Impact on the Architecture

Feedback about realisation effort

For the stakeholder, it is important to get feedback about the consequences of the deltas on the realisation effort. The realisation effort of a particular variability model delta and the respective requirements artefact deltas are evaluated with respect to:

- The adaptation effort for the product line architecture.
- The realisation effort for the components.
- The maintenance effort

Categorisation of architecture changes

The variability model delta and the associated requirements artefact deltas are provided to the application architect who maps them to the reference architecture using the traceability links established between domain requirements artefacts and domain design artefacts in the domain engineering process.[43] The adaptation effort for the reference architecture is roughly estimated by determining the category into which the architectural changes (e.g. changes of interfaces, the structure or the texture of the architecture) fall. For a more detailed estimation, the realisation effort for components has to be taken into account as well.

We distinguish between the following four categories of architectural adaptation:

- *Category A – No adaptation effort*: A particular variability model delta belongs to this category if no adaptation of the reference architecture is needed. In other words, the realisation of the delta has no impact on the architectural structure and texture.

  No impact on architecture

- *Category B – Moderate adaptation effort*: A particular variability model delta is assigned to this category if only local architectural adaptations are required, e.g. changes to single components. Deltas that lead to slight adjustments of cross-cutting aspects belong to this category as well. This includes the adjustment of design quality requirements and simple changes of the architectural texture (but no significant change of architectural structure).

  Local impact, slight adjustments

- *Category C – High adaptation effort:* Variability model deltas lead to a high adaptation effort if the reference architecture needs global changes. A change is considered to be global if, for example, a significant number of components and/or interfaces have to be changed.

  Global impact

- *Category D – Too high adaptation effort*: A variability model delta that falls into this category means that no economically reasonable realisation of the delta is possible within the software product line. The developing company has to reject this delta unless the option exists to realise the desired application in a separate development project.

  Out of product line scope

[43] The mapping of requirements to architecture ensures a consistent integration of changes. A scenario-based approach for mapping requirements to the architecture of a software product line is presented in [Pohl et al. 2001a].

Table 15-1: Relation between deltas and architectural adaptation effort categories

Category / Delta	Category A **No** adaptation effort	Category B **Moderate** adaptation effort	Category C **High** adaptation effort	Category D **Too high** adaptation effort
1. New variant	X	X		
2. Adaptation of variability dependencies	X	X	X	
3. Adaptation of constraint dependencies		X	X	X
4. New variation point		X	X	X

Example 15-8: *Examples of Architectural Adaptation Effort*

- *Effort category B (delta 1)*: In case the stakeholder has special quality requirements with respect to a particular variant (e.g. performance requirements) this may lead to a moderate adaptation of the reference architecture.
- *Effort category A (delta 2)*: A change from "optional" to "alternative choice" may lead to essentially zero adaptation effort, because it is mainly a matter of configuration, i.e. configurations that include both variants have to be prohibited.
- *Effort category D (delta 3)*: The deletion of an "exclude" dependency might lead to a very large change in the architectural structure, e.g. if the dependency exists for technological reasons.
- *Effort category D (delta 4)*: A new variation point leads to too high an adaptation effort if the reference architecture is not able to support the required external variability. This may happen, for example, if the domain artefacts support sequential processing, yet to realise the delta, a variation point with the two alternative variants 'sequential processing' and 'parallel processing' is necessary. In this case all components would have to be reengineered to enable parallel access and synchronisation.

Example 15-9: *Trade-off Decision*

The stakeholder requires the following functionality: "The home owner shall be able to send a request about the door lock status via mobile phone." Yet, the product line provides the presentation of the door lock status only via the Internet. Now the stakeholder has the following alternatives:

1. Insisting on the mobile phone solution. Then the development organisation may realise the requirement as described above. This decision causes realisation effort that depends on the assignment to one of the described categories.
2. Accepting the Internet solution and giving up the requirement regarding the mobile phone. Then the stakeholder requirement is adapted (and is satisfied by the domain requirements artefact). The realisation effort will be very small because this stakeholder requirement can be fulfilled completely by reusing domain artefacts.

Assignment of deltas to effort categories

Table 15-1 shows the basic relations between the types of variability model deltas introduced above and the four categories of architectural adaptation effort. Variability deltas are depicted as rows in the table. Each column represents a category of architectural adaptations. A grey-filled table cell with an "X" indicates that the corresponding delta is likely to fall within the particular adaptation effort category.

Decision about requirements

When the stakeholders (and the application requirements engineer) get feedback from application design about the estimated realisation effort for a variability model delta and/or a requirements artefact delta, they have to decide between the following alternatives:

Realisation of the delta

- The application variability model and/or the application requirements artefacts are adapted to fulfil the stakeholder requirement. In other words, the delta is realised.

Reuse of domain requirements artefacts

- The stakeholder adapts the requirement (which means the stakeholder might only get an 80% solution).[44] In this case no additional implementation is needed and the domain artefacts can be reused without changes.

Partial realisation of the delta

- The stakeholder requirement as well as the application variability model and/or the application requirements artefacts are adapted, i.e. the stakeholder requirement is partially fulfilled.

[44] An 80% solution represents an application that fulfils most of the stakeholder's needs but involves certain compromises about stakeholder requirements on the one hand and realisation effort on the other hand.

Removed stakeholder requirement

- The stakeholder decides that the requirement should not be realised due to the high adaptation effort (see the second alternative in Example 15-2).

Support of trade-off decisions

In the case of a high adaptation effort, but also in other situations, the orthogonal variability model and the associated requirements artefacts provide assistance in finding an acceptable trade-off solution. The variability model presents options that can be chosen instead of realising the original stakeholder requirement. The stakeholder can select the best possible solution provided by the product line. Example 15-10 illustrates the support of a trade-off decision.

> ***Example 15-10:*** *Support for Trade-off Decisions*
>
> The variability model depicted in Fig. 15-10 contains the variation point 'home security by'. The stakeholder requirement under consideration demands outdoor intrusion detection via photo electric guard. The product line provides the feature 'outdoor motion detection'. Thus, a delta between the stakeholder requirement and the domain artefacts occurs. During the discussion with the stakeholder about the estimated realisation effort of this delta, the application requirements engineer uses the variability model to show the stakeholder possible alternatives. The engineer presents two alternatives with regard to home security: 'camera surveillance' and 'motion detection'. The description of the corresponding features may cause the stakeholder to resort to one of these alternatives.

15.5 Documentation of the Application Requirements

Contents of the specification

The results of the activities described in Sections 15.3 and 15.4 have to be documented in the application requirements specification. The specification is the basis for the other application engineering sub-processes and defines all application requirements. The application requirements specification includes:

Reused requirements

- *The application requirements artefacts that correspond to domain requirements artefacts including the traces to the respective domain requirements artefacts*
  This part of the documentation contains all application requirements artefacts that are reused without adaptations. It consists of the commonalities and the bound variation points, i.e. variation points that are bound to the selected variants.

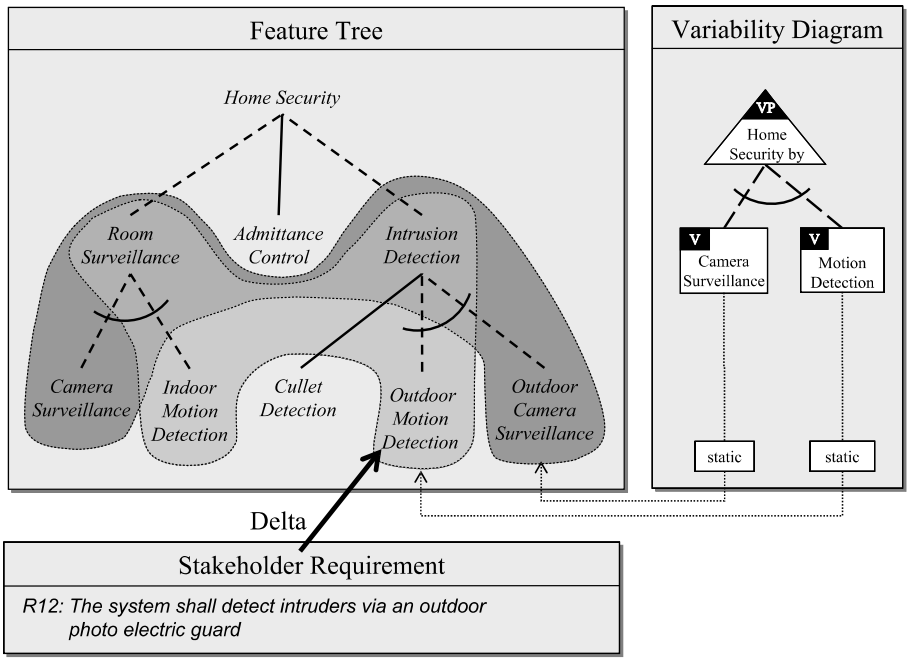


Fig. 15-10: Example of supporting trade-off decisions

- *The application variability model* — *Application variability model*
  This part of the documentation describes the application variability model with the selected variants, i.e. with the variability bindings.
- *The variability model deltas including the traces to the original variability model elements* — *Application variability model deltas*
  Variability model deltas are included in the application variability model.
- *The relation of the application requirements artefacts to the selected variants in the application variability model* — *Traces to selected variants*
  This part of the documentation captures the traces between the requirements and the variants that are selected for the application.
- *The requirements deltas including the traces to the original domain requirements artefacts* — *Requirements deltas*
  This part of the documentation contains all requirements artefacts that are new or have been modified specifically for the application.

Traceability

The traces to domain requirements artefacts contained in the application requirements specification are an essential means to support the reuse of domain artefacts in application design, application realisation, and application testing. The bindings of the variability in the application requirements

artefacts provide the basis for deriving the bindings for the variation points in the reference architecture, in the components, and in the test artefacts. The traces of changes to the variability model enable, for example, the application architect to refine the variability model deltas with respect to architectural variability. The traces from modified requirements artefacts to the original domain artefacts enable the application architect to identify the corresponding artefacts of the reference architecture and adapt them accordingly.

15.6 Differences from Single-System Engineering

Requirements engineering activities

Requirements engineering in single-system development encompasses the elicitation, validation, negotiation, and documentation of requirements (see e.g. [Pohl 1997]). In addition, continuous requirements management has to ensure that the specified requirements are always up to date.

Specific activities

The application requirements engineering sub-process in software product line engineering includes four major activities that differ from requirements engineering activities in single-system engineering:

Communication of variability

- *The communication of external variability to stakeholders*
  The goal of this activity is to communicate the variation points, variants, and associated requirements to the stakeholders. The results of this activity are a set of variants that have to be bound for the considered application and a set of deltas between the application requirements artefacts and domain requirements artefacts.

Deltas

- *The evaluation of the realisation effort for requirements deltas*
  The goal of this activity is to evaluate the impact of deltas on the variability model, the requirements artefacts, the reference architecture, etc. Based on the estimated effort, the stakeholder decides whether a delta should be realised or not.

Traceability

- *The documentation of application requirements*
  The goal of this activity is to define the application requirements and to record traceability links between the domain requirements artefacts and application requirements artefacts. The result of this activity is the application requirements specification.

Variability bindings

- *The documentation of variability bindings*
  The goal of this activity is to document the bindings of the variation points defined in the domain variability model. The result of this activity is the application variability model.

15.7 Summary

Reuse of domain requirements artefacts

The goal of application requirements engineering is to elicit stakeholder requirements for the application and map the stakeholder requirements to common and variable domain requirements artefacts. Thereby domain requirements artefacts should be reused as much as possible. The reused common and variable domain requirements artefacts become part of the application requirements specification. The bindings of the variation points defined in the domain variability model are documented in the application variability model.

Realising deltas

If the stakeholder requirements for the application cannot be satisfied by reusing common or binding variable domain requirements artefacts, application-specific requirements artefacts may be introduced. Since these artefacts differ from the domain requirements artefacts, so-called requirements deltas arise. In addition, the application variability model may be adapted leading to deltas between the application variability model and the domain variability model. Application architects estimate the effort required for realising the application-specific requirements. This estimation is taken into account for deciding if the deltas are realised in the application or not. If it is decided to realise the deltas, the application requirements engineers adapt the application requirements artefacts and/or the application variability model to satisfy the stakeholder requirements.

Frank van der Linden

16 Application Design

In this chapter you will learn:

- *About the interrelations of the application design sub-process with the application requirements engineering, domain design, and application realisation sub-processes.*
- *How to derive an application architecture from the product line reference architecture based on the application requirements.*
- *How to bind variability in the reference architecture.*
- *How to determine realisation costs of adaptations of the domain artefacts required for an application.*

16.1 Introduction

Goal of application design

The main goal of the application design sub-process is to produce the application architecture. The application architecture is a specialisation of the reference architecture developed in domain design. Application architects bind the variability of the reference architecture and introduce application-specific changes according to the application requirements specification. The application architecture is passed on to application realisation where the reusable components and interfaces are assembled and where application-specific components and interfaces are developed. The sub-processes and artefacts closely related to the application design sub-process are highlighted in Fig. 16-1. The major information flows between application design and its related sub-processes are shown in more detail in Fig. 16-2.

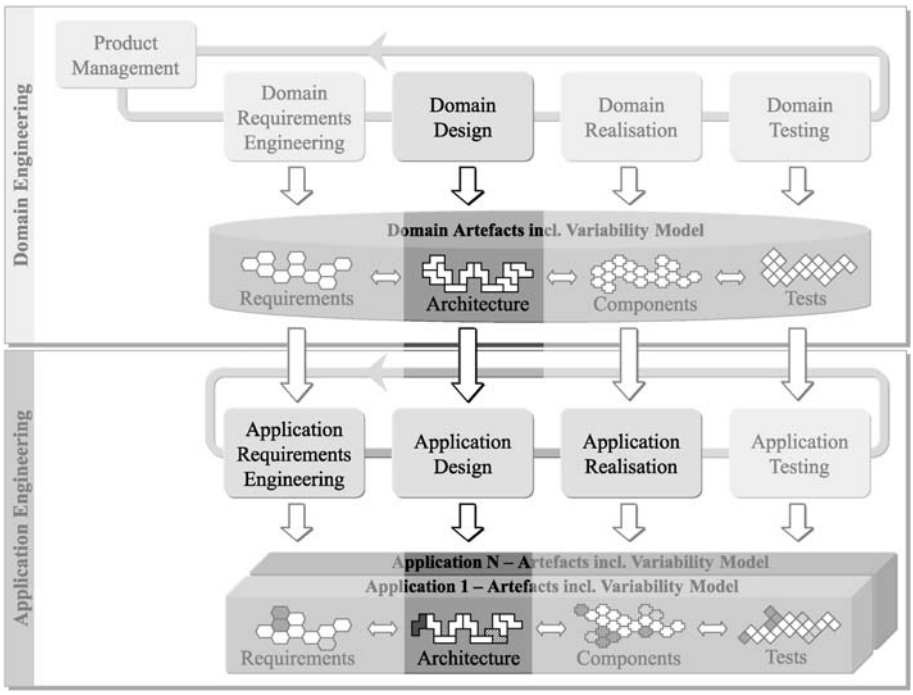


Fig. 16-1: Sub-processes and artefacts related to application design

16.1.1 Interrelation with Application Requirements Engineering

Application requirements

Application requirements engineering is responsible for developing the application requirements specification. The specification includes the applica-

tion variability model as well as all application requirements, including the requirements reused from the domain artefacts and the application-specific requirements. It also contains traceability links to domain artefacts as well as the relation to selected variants in the variability model. The entire application requirements specification is passed on to application design (① in Fig. 16-2).

Realisation effort

Application design supports trade-off decisions made in application requirements engineering by determining the estimated realisation effort (② in Fig. 16-2). The trade-off decisions about application-specific requirements are part of the negotiation with stakeholders in the application requirements engineering sub-process.

16.1.2 Interrelation with Domain Design

Reference architecture

Domain design develops the reference architecture, which is the basis for the application architecture. The application architect binds the architectural variability according to the bindings defined in the application variability model. The reusable domain artefact selection indicates the reusable domain artefacts (③ in Fig. 16-2).

Feedback from domain design

The application architect provides feedback through requests for additional and altered design (first bullet of ④ in Fig. 16-2) that may lead to an improvement of the reference architecture. Furthermore, application design delivers design artefacts, which have to be reengineered for flexibility and reusability and incorporated into the platform, to domain design (second bullet of ④ in Fig. 16-2).

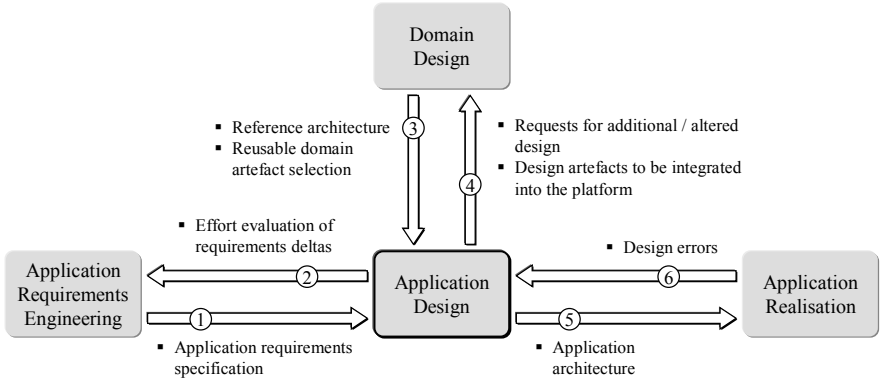


Fig. 16-2: Information flows between application design and other sub-processes

16.1.3 Interrelation with Application Realisation

Application architecture

Application realisation builds the application based on the application architecture (⑤ in Fig. 16-2). The application architecture determines the structure of the application to be built as well as the rules how to build it, which are contained in the texture. The application architecture also determines the configuration of reused domain components and interfaces that are part of the application as well as their interrelation with application-specific components and interfaces.

Design errors

Application realisation provides feedback on all kinds of design errors (⑥ in Fig. 16-2) that emerge during realisation and have to be solved by the architects. These include, amongst the normal design errors, components and interfaces that turn out not to be reusable, and configurations that do not work properly.

16.2 Development of the Application Architecture

Specialisation of the reference architecture

An application architect has similar responsibilities as a traditional architect. As such, abstraction, modelling, simulating, and prototyping are activities that are performed by the application architect. However, all those activities have to be performed only for the application-specific parts. The reference architecture includes a lot of decisions that can be reused in application engineering. The application architect starts with the reference architecture and specialises it towards the application architecture. The reference architecture models are specialised through the binding of variants according to the bindings in the application variability model and by adding application-specific parts.

> ***Example 16-1:*** *Application-Specific Abstractions*
>
> The home automation example employs domain abstractions for authentication such as “authentication key” and “authentication algorithm”. A new application is planned that supports iris scan authentication, a feature that is not yet supported by any other product in the product line. Consequently, there are no abstractions available for iris scan authentication. The architect adds new abstractions such as “iris map” and “iris pattern” to support the new feature.
>
> The platform provides abstractions to deal with quality features, such as safety, security, and performance. The new application needs preventive maintenance of hardware parts that fail frequently. The application architect thus provides abstractions dealing with maintenance, e.g. “error level” and “error rate”.

16.2.1 Application-Specific Modelling

Application-specific abstractions

The application architect introduces abstractions that are necessary for the specific application at hand, i.e. the architect adds abstractions for application-specific aspects that are not covered by the domain artefacts. The additional abstractions are usually related to application-specific requirements. Especially when there are very strict quality requirements, new application abstractions have to be introduced. For instance, if performance requirements of the application are stricter than defined in the product line, the application architect adds abstractions for threads' synchronisation and communication. Also in the case where the application supports a feature that is not provided by the product line, abstractions related to the new feature are added (Examples 16-1 and 16-2). The abstractions of the application architecture have to be integrated with the abstractions defined in the reference architecture to obtain a consistent application architecture.

> ***Example 16-2:*** *Lock Control Application Design Activities*
>
> An application is planned for a mid-range system, including both a sliding door and an iris scan identification feature. Both are new features, and the application is the first one that has to support them. In addition, the application has a normal swinging door and the basic keypad commands for situations without iris scan. For instance, during the recognition phase of a specific iris the keypad is needed. The application architect provides the necessary abstractions of both new features and relates them to the abstractions in the reference architecture dealing with door control and authentication. The models get new elements related to the abstractions of the reference architecture. There are components or plug-ins for 'sliding door lock actuator', 'sliding door open/close sensor' and 'iris scan authentication'. A thread is added to perform the iris scan authentication. In order to ensure that this thread does not interfere with the other threads, an existing application is enriched with a mock-up iris scan algorithm that occupies the processor and the memory for one-half to three-quarters of a second and answers "no" or "yes" randomly. This system is used to simulate the additional thread.

Application-specific models

The application-specific abstractions have connections to the concepts and models defined by the reference architecture. Application-specific models are built for establishing and dealing with these connections. Additional models are built to accommodate application-specific concerns, such as specific behaviour or specific quality requirements.

Simulation and prototyping

As the application architecture is more concrete than the reference architecture, more realistic simulations can be performed. Application architects employ simulations to get an insight into the properties of a specific configuration and to evaluate application-specific variants (see Example 16-2). For example, performance simulations can be used to determine the configuration that satisfies the application-specific performance requirements best. As for simulation, prototyping can be more concrete at the application level, and relate to application-specific requirements.

16.2.2 Binding of Variants

Variants of reference architecture

The application architect has to bind the variants for the variation points of the reference architecture as defined in the application variability model. The quality of the reference architecture determines whether this is an easy job. It depends on:

- The way mass customisation is incorporated; this determines which styles, structures, and patterns are used to deal with variability.
- The abstractions used, determining which variation points and variants are available.
- The traceability between variability in domain requirements artefacts and the reference architecture.

> ***Example 16-3:*** *New Variants in the Application Architecture*
>
> The design of the door and window management subsystem for the considered application has to include additional functionality for sliding doors and for iris scan identification. The application architect reuses the keypad and swinging door functionality. For the new features, new components have to be made; see Fig. 16-3. For the sliding door feature, two application-specific plug-in components are needed: a ‘sliding door lock actuator plug-in’ component and a ‘sliding door sensor plug-in’ component. Note that the basic lock functionality does not need a door sensor. For the iris-scan feature, two plug-ins are needed: an ‘iris control plug-in’, which records the iris scan to initiate commands, and an ‘iris authentication plug-in’, which performs the actual authentication algorithm. Interfaces of the reference architecture are reused to connect the new components to the remainder of the door functionality. No new interfaces are needed, and no new patterns or frameworks either.

Reusable domain artefacts

To reduce the work of application realisation as much as possible, reusable domain artefacts have to be used whenever possible. The reference architec-

ture determines common components and interfaces. By binding the variation points, the application architect selects additional domain design artefacts that can be reused. If no domain artefact is available, the application architect has to define an application-specific one to be realised during application realisation.

Design artefacts for new variants

Certain variants have to be realised only for one single application. These variants may involve new components (Example 16-3) or interfaces, but sometimes also larger parts of the structure, such as configurations of components and interfaces, new patterns (Example 16-4), and even new component frameworks.

Example 16-4: *New Patterns*

An application is specified that uses Bluetooth [Bluetooth 2004] to connect a wireless device for all kinds of user interaction. Since the product line does not yet have any wireless communication, several components have to be added to the application architecture. This also involves the introduction of new interfaces between them, and of corresponding patterns. For instance, there is a pattern involving generic Bluetooth functionality with plug-in components for several specific input devices. This pattern has to be integrated with the existing components dealing with user control.

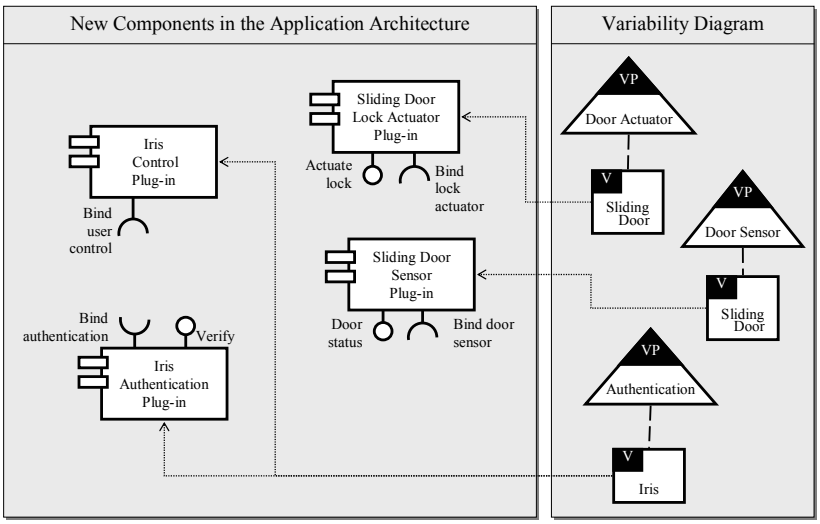


Fig. 16-3: New variants in lock control

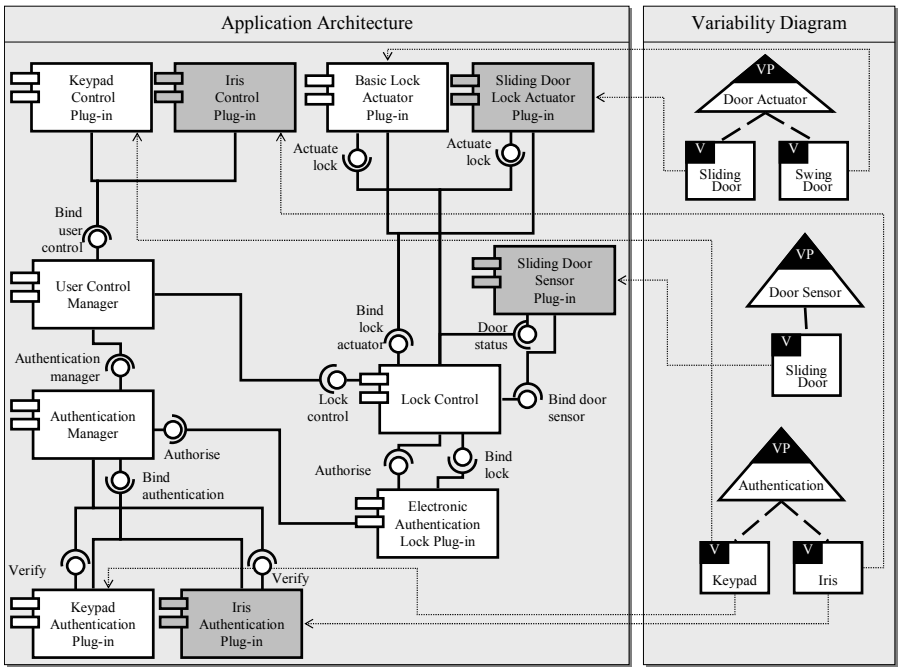


Fig. 16-4: Lock control application with sliding doors and iris scan

Realisation effort

When designing the application-specific parts, the architect has to consider the additional effort in application realisation. Often, there are not enough personnel and time available to realise many specific components from scratch. The architect has to evaluate carefully what can be implemented specifically for a single application with regard to the available resources. The amount of available resources for realising the application has to be negotiated with application requirements engineering, the stakeholders, and product management. For normal application development, the amount of resources may be scarce, which means that not much additional work can be done. In applications that are of strategic relevance, e.g. if the application is a lead product, or the client is the most important client, additional resources may be allocated to the project. Section 16.4 treats this topic in more detail.

16.2.3 Determining the Configuration

Variants determine required components

The specific configuration of components in an application is the result of the binding of the variation points with the selected variants. Certain domain components are not required in the application and therefore are absent. If different variants that are realised in different plug-in components are simultaneously present in the application, more than one plug-in is needed. Application-specific components are designed as additional plug-ins, if possible.

In this case they are connected at the places that are planned for them in the reference architecture.

> ***Example 16-5:*** *Lock Control Application Variants*
>
> Figure 16-4 shows a part of the application architecture of the new application with both an additional sliding door and iris scan identification. Reusable domain components are depicted as white boxes. New application-specific components are depicted as grey boxes. These are the same as in Fig. 16-3. The configuration resembles the domain structure, but certain plug-in components are present twice, once for the basic functionality and once for the additional, new features. Moreover, the plug-in for manual door locks is absent. Basic functionality is provided by the standard, reusable domain plug-in components. The 'sliding door lock actuator plug-in' component is put in parallel[45] with the 'basic lock actuator plug-in' component. The 'sliding door sensor plug-in' component is the only door sensor plug-in since, for the basic lock functionality, a door sensor is optional and is not used in this application. The two plug-in components, 'iris control plug-in' and 'iris authentication plug-in', are put in parallel with the plug-in components for keypad authentication.

16.2.4 Consistent Selection of Component Variants

Global impact of variants

The application architect is in charge of deciding about the variants that have global consequences on the application. As the information about variants is distributed over the different components, care has to be taken to select a consistent configuration of component variants.

Hardware-specific configuration

Part of the product line variability deals with the hardware specifics of the application, such as memory size and amount of peripheral hardware. Hardware-specific variations are either bound by selecting the appropriate components or by setting the configuration parameters of one or more components to the proper values.

> ***Example 16-6:*** *Hardware-Related Variability*
>
> Each home automation system has a specific set of sensors and actuators in a specific configuration with other hardware, such as routers or switches. The application uses one of several pre-selected protocols, each suitable for a specific network configuration. Finally there is a limited memory size and bandwidth available.

[45] This means that it is connected to the same interfaces of other components, and both are present together.

Example 16-7: *Consistency of Component Variants*

The application with the sliding door and iris scan has a single sliding door and two swinging doors. Each swinging door needs one actuator. The sliding door needs two, one at the top and one at the bottom. Several variants are selected based on this hardware configuration:

Components	*Variant*
Lock control	map of three doors to four door lock actuators
Door lock plug-ins	three doors, their properties, and the identification of their authentication algorithms
Actuator	activation and control of each of the four actuators separately

Application design uses one or more configuration files to keep such configuration information consistent. Application realisation has to use the information in these configuration files.

16.3 Feedback of Application Artefacts to the Domain

Application as test bed for the domain

Variants that are designed by the application architect may be usable for other applications as well. This also holds for other technical solutions provided by the application architect. The application architect is in discussion with the domain architect to identify such possible reusable artefacts, which may have a wider scope than the present application. In many cases the artefacts are first produced for a single application. This is a test bed for the domain architect. If everything works fine and product management decides that the new artefacts should be integrated into the domain artefacts, the domain architect takes over, and takes care of a redesign to make the artefacts reusable (Example 16-8).

Example 16-8: *Integration of Application Artefacts into the Platform*

Since the application with sliding doors and iris scan is a commercial success, the programme manager asks to allow the corresponding requirements to be reused in other applications. They are incorporated as variants of existing domain requirements artefacts for door control and authorisation. The domain architect takes over the application architecture for these parts, provides a redesign of the application design artefacts, and initiates adaptation during domain realisation.

Replacing non-reusable artefacts

After an application artefact has been successfully integrated into the platform, the application architect may decide to use the reusable artefact in new versions of the application instead of the non-reusable application-specific artefact which was originally built for this application. This reduces the amount of application-specific artefacts that have to be maintained.

16.4 Effort and Cost of Variants

Realisation cost

The cost of selecting a variant for the application architecture depends on what has to be done to realise the variant in the application. If the variant is realised in the platform, the cost is small (category A, see Section 15.4.4). The main cost and effort are caused by determining the right parameters. In the case where application-specific components have to be developed, the costs are typically much higher. When the application-specific components provide and use only domain-specific interfaces, the development cost and effort are still moderate (category B, see Section 15.4.4). In many cases, application realisation can use other components as templates for the application-specific components. In the case where a large new part of the structure has to be provided, costs and effort are typically high (category C, see Section 15.4.4). In that case, new interfaces have to be provided, which have to be realised as well. In addition, simulation and validation have to be performed before the application architecture is finished. In cases where large parts of the architecture have to be added, e.g. the introduction of a new framework supporting a new (variant of a) feature, the changes may be very large, to such an extent that they cannot be realised using the reference architecture (category D, see Section 15.4.4).

Cost factors

The following factors influence the required effort and costs:

- Number of new components to be realised.
- Number of interfaces to be realised.
- Number of small component and interface adaptations – care has to be taken that large component and interface adaptations cost almost the same as, or even more than, writing a new one.
- Simulations performed to verify the correct behaviour of adaptations and extensions – the need for simulation indeed depends on the kind of changes, but when new quality requirements are involved simulation is often inevitable.
- Adaptations to cross-cutting aspects – this may require adaptations to all, or many, components. It may require the addition of new interfaces to be provided or used by many components. Typically this also involves costs of simulations or other ways to verification.

- Tests to be performed on reused components and configurations.
- Tests to be performed on new components – this takes more effort, since new test cases have to be created.

Cost estimation

To estimate the effort and costs the organisation may have standard figures that apply for most situations. In specific situations, the organisation may have to adapt the standard costs, e.g. if a change does not differ much from another one, the costs may be smaller. If a complete, new design has to be introduced, costs may be estimated higher. The cost of introducing a new variant is communicated to the requirements engineers and other stakeholders, who, in the end, decide whether the costs are worth the value they bring. If the costs are too high, adaptations of the application requirements may lead to affordable development costs.

Cost amortisation for reusable features

In cases where the application is used as a test bed for a new feature, additional costs may be acceptable to the organisation since the new feature can also be reused within future applications of the product line.

16.5 Differences from Single-System Engineering

The application architect has to specialise the reference architecture to an application architecture for a single application. This means that:

Reuse of reference architecture

- A large part of the application architecture is determined by the reference architecture and does not have to be designed by the application architect. However, the reference architecture may be under-specified, meaning that application-specific artefacts have to be designed by the application architect.
- Variation points in the reference architecture are bound to application-specific variants according to the bindings of the variability defined in the application variability model. Thereby domain design artefacts are reused.

Traceable requirements

- Many application-specific requirements are specialisations of domain requirements. This gives a first indication of where the architecture has to be specialised.

Common rules from texture

- The texture in the reference architecture not only captures the commonality within a single system but also defines commonality that is present within all applications. The application architecture must conform to the texture. Texture has an additional role in providing common ways to deal with variability in the application architecture.

- Software product line engineering means that the activities of the application architect require less effort than in single-system software engineering. If no application-specific adaptations are required, the application architecture can be established by binding the defined variation points with predefined variants.

Reduced effort

16.6 Summary

Large-scale reuse

Application design has the same role as single-system software design. The application architecture determines the overall structure of a particular application and must be capable of satisfying the application requirements. The application architect uses the reference architecture, which provides a design for many of the application requirements that the application architect must satisfy. Moreover, reusable components and interfaces, and configurations of them, are provided by the reference architecture. Therefore, application architects can focus their attention on the application-specific parts, thereby saving a lot of time.

Source of future domain artefacts

In discussion with the domain architect, certain solutions and application artefacts may become candidates for integration into the platform. Usually the integration takes place after the application is finished and the properties of the developed artefacts are validated. The application architect has the responsibility to provide the domain architect with information about such possible artefacts.

Frank van der Linden

17

Application Realisation

In this chapter you will learn:

- *About the interrelations of the application realisation sub-process with the application design, domain realisation, and application testing sub-processes.*
- *How to bind the variability according to the application design and the application requirements.*
- *How to derive application-specific components and interfaces from the application architecture and thereby guarantee a high degree of reuse.*
- *How to realise a consistent application configuration.*

17.1 Introduction

Goal of application realisation

The goal of application realisation is to develop applications that can be tested and brought to the market after ensuring a sufficient quality. Application realisation provides the detailed design and implementation of application-specific components and configures them with the right variants of the domain assets into applications. The main results of application realisation are the application-specific components and interfaces, the selected variants of reused components, and the application configuration. The sub-processes and artefacts closely related to the application realisation sub-process are highlighted in Fig. 17-1. The main information flows between application realisation and its related sub-processes are depicted in Fig. 17-2.

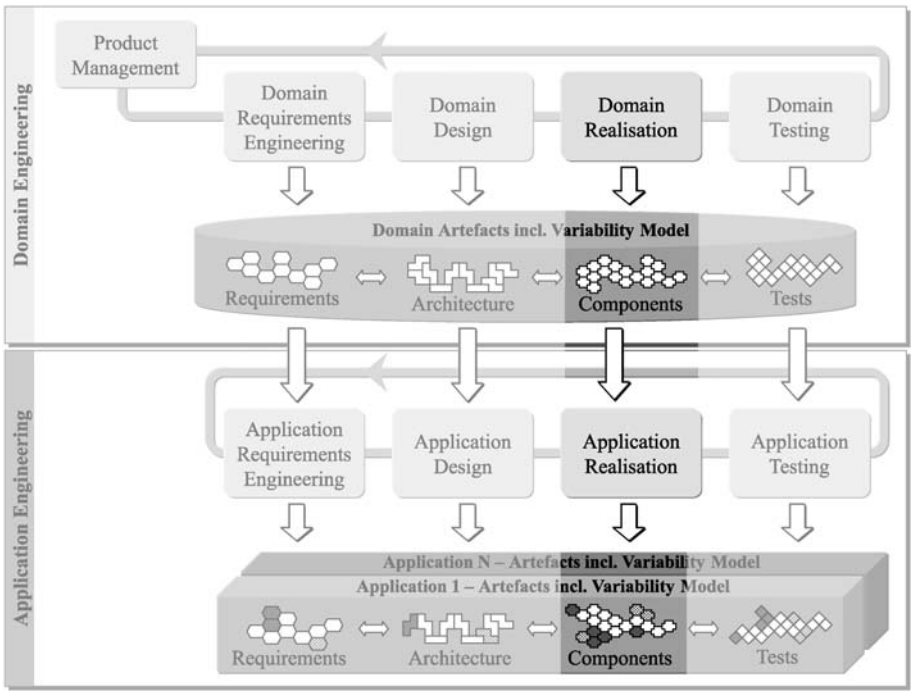


Fig. 17-1: Sub-processes and artefacts related to application realisation

17.1.1 Interrelation with Application Design

Application architecture

Application design provides the application architecture which determines the configuration of components and interfaces, which are either reused from the platform or designed specifically for the considered application (① in Fig. 17-2).

Design errors

Application realisation provides feedback with regard to all kinds of design errors (② in Fig. 17-2) that are uncovered during realisation and which have to be corrected by application design. An example of such an error is a configuration that does not work properly.

17.1.2 Interrelation with Application Testing

Executable application

Application realisation delivers a complete application (first bullet of ③ in Fig. 17-2), ready for testing. Application testing performs unit, integration, and system tests based on the application requirements, design, and realisation artefacts (i.e. the detailed design of components and interfaces). Application realisation supports application testing by providing the interface descriptions (second bullet of ③ in Fig. 17-2) required, for instance, for the unit test.

Test results

As feedback, application realisation gets test results including the acceptance or rejection of the application and problem reports describing in which way the test item fails (④ in Fig. 17-2). The feedback is used to improve the application until acceptance is reached. Moreover, application testing reports defects in interface descriptions which are detected in application testing as they hamper the development of test cases.

17.1.3 Interrelation with Domain Realisation

Reusable components and interfaces

The input for application realisation from domain realisation consists of reusable components and interfaces (⑤ in Fig. 17-2) designed, implemented, and ready for reuse. In order to be able to integrate these artefacts into an application, domain realisation additionally provides configuration support.

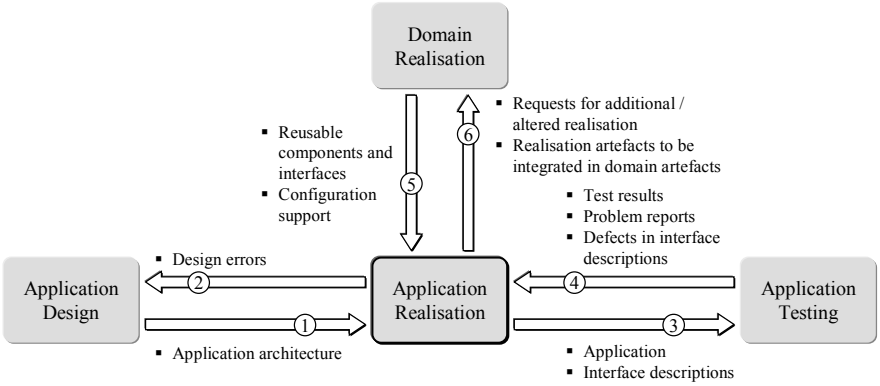


Fig. 17-2: Information flows between application realisation and other sub-processes

Feedback to domain realisation

Application realisation provides feedback through requests for additional and altered realisation (first bullet of ⑥ in Fig. 17-2). The requests pertain to functionality or quality that should be provided by the platform. Furthermore, application realisation develops application-specific components and interfaces that may possibly be integrated into the domain artefacts (second bullet of ⑥ in Fig. 17-2).

17.2 Configuration

Domain realisation delivers components and interfaces for reuse by application realisation. Interfaces are reused without changes, but components often have internal variation points, which have to be bound.

Use of domain interfaces

Many interfaces between the components of an application are reusable domain interfaces. Most of the application components carry them as they are an important means to support the texture and to implement variation points. Different variants of a single component often require and provide the same interfaces. Plug-in components provide the interfaces required by a particular component framework. Moreover, the architectural texture may demand that each component provides aspect-specific interfaces (Section 11.5). Such interfaces are realised during domain engineering but are heavily used during application engineering.

Reusable component variants

Reusable domain components often have internal variation points. Domain realisation provides mechanisms to support variant selection. Variants within a component can be selected, for instance, by parameter bindings. Guided by the application architecture and the application variability model, application realisation selects the proper variants of the components to be part of the application. For each reusable component, application realisation determines the right choice of component parameters to bind the required variant.

> ***Example 17-1:*** *Lock Control Component Reuse*
>
> Suppose that the application has one sliding door with two actuators and one sensor, and two swinging doors, each with one actuator but no sensor. The ‘lock control’ component is reused. It has to be configured in such a way that each of the doors is controlled using the right sensors and actuators.

Parameter consistency

In many cases, several components need to know the same kind of information. This may result in a lot of redundancy in the set of parameters. Human error in providing the parameters may lead to inconsistencies. Application realisation has to keep track of the parameters and their relationships in order to keep the configuration of components consistent. The quality of the refer-

ence architecture and the resulting component designs determine whether it is a simple or a complex task to select component parameters consistently. To reduce the redundancy of parameters, domain design and realisation may introduce configuration components or employ tool support [V. Ommering et al. 2000].

Example 17-2: *Binding Variation Points in Components*

The application with sliding doors and iris scan reuses many platform components and interfaces, e.g. the 'user control manager', 'authentication manager', and 'lock control' components of the door control framework (Fig. 17-3). They have to be configured to be able to deal with the specific situation of having both keypad and iris scan, two swinging doors, and one sliding door. Domain realisation provides them in such a way that they get the necessary configuration information from plug-in components that are bound to them. The parameter information does not have to be provided twice. Certain plug-in components are reused as well:

- 'Keypad control plug-in' and 'keypad authentication plug-in' are reused by many applications. Their variants are chosen according to the authentication-key formats, e.g. four digits. Moreover, there may be variants that differ in the handling of situations when too few or too many keys are pressed. Since those components and key formats are already used by other applications, the configuration information can be reused.
- 'Electronic authentication lock plug-in' is one of the six kinds of reusable lock plug-in components defined in Example 12-3. It needs to be configured to be able to deal with two different authentication algorithms. The specific parameters of these algorithms are available in the respective components that deal with these algorithms. The main function of the component is to select the right algorithm in the right situation.
- 'Basic lock actuator plug-in' is reused in many applications with the same kind of swinging doors. For this component, configuration parameters like timing and speed have to be provided.

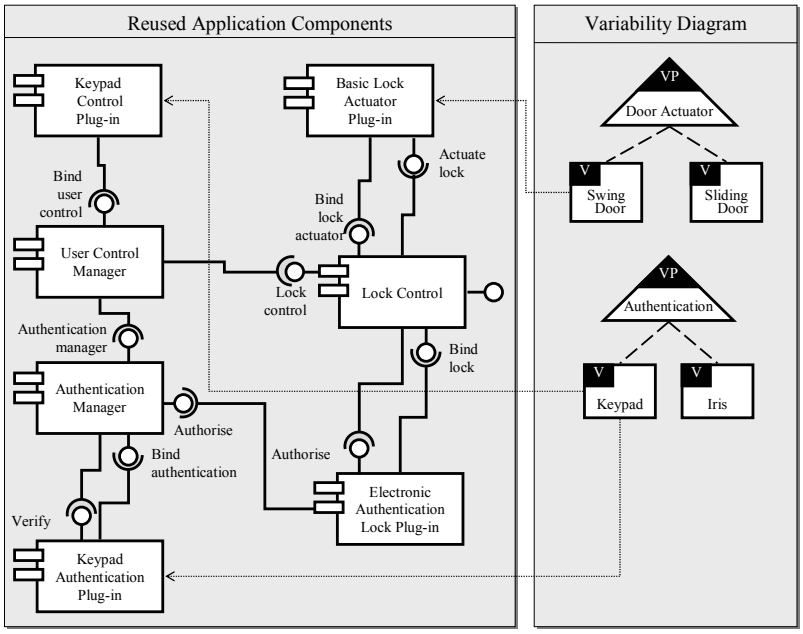


Fig. 17-3: Reused domain components and interfaces

17.3 Realisation of Application-Specific Components

Application-specific components are realised just as in single-system engineering. However, in many cases, domain interfaces can be reused for application-specific components. For instance, an application-specific plug-in component has to carry all interfaces that are determined by the domain architecture for such a plug-in.

Reusable component adaptations

Application-specific components and interfaces are needed whenever there is no suitable reusable domain component available. Making application-specific components reusable is not of interest for the application developer whose focus is on a single application only. If the component has to be integrated into the domain artefacts, domain realisation takes over at an appropriate time, not disturbing application realisation. However, an application component may resemble already existing domain components. For instance, it may be the case that the application component and some reusable components are variants of a common variation point. In that case, it is a good idea to use the design of the existing variants as input for the design of the new variant.

Example 17-3: *Application-Specific Components for Lock Control*

The application architecture is available for an application that includes both a sliding door and an iris scan identification feature. Since there are no reusable components available for these features, the application architect determines new components for ‘sliding door lock actuator plug-in’, ‘sliding door sensor plug-in’, ‘iris control plug-in’, and an ‘iris authentication plug-in’; see Fig. 17-4. All these components reuse existing interfaces:

- ‘Bind user control’
- ‘Bind lock actuator’ and ‘actuate lock’
- ‘Bind door sensor’ and ‘door status’
- ‘Bind authentication’ and ‘verify’

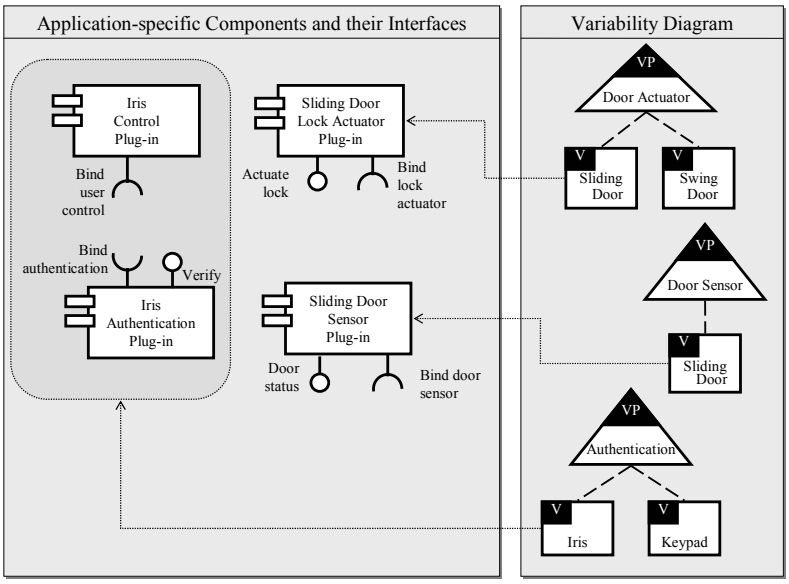


Fig. 17-4: Application-specific components

17.4 Building the Application

The final task of application realisation is the realisation of the configuration that is actually delivered as the application. Component variants have to be

compiled, linked, and deployed on the actual hardware. In all these steps variability may be bound, depending on the configuration mechanism used.[46]

Files for components and interfaces

A component variant is realised as a collection of files. The files encompass source code files, header files, and parameter definitions. Interfaces are usually realised in one or more header files. The header files for required interfaces are necessary to be able to compile the component variant. Note that the provided interfaces do not need to be included themselves. Linking combines the component variants to executables or dynamic link libraries. The application is made up of one or more executables and dynamic link libraries.

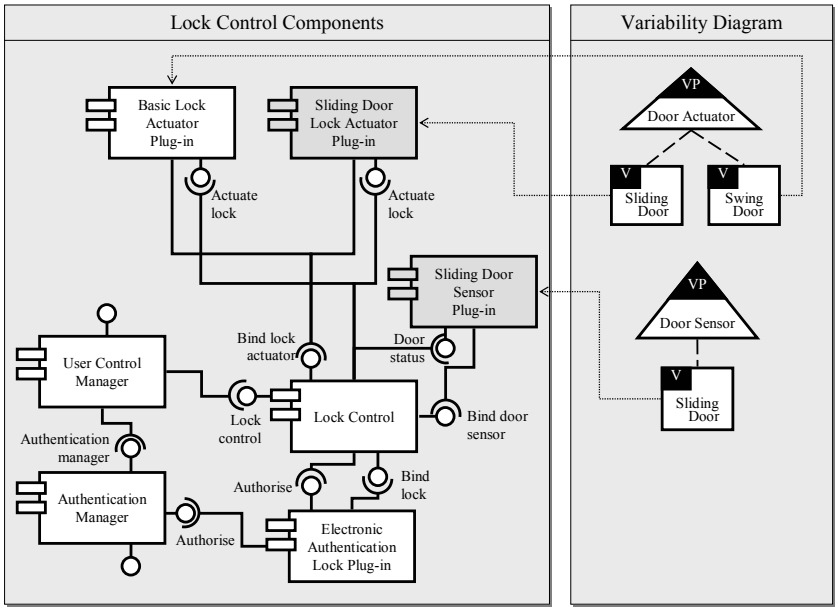


Fig. 17-5: Lock control configuration

Versions and variants

As in single-system engineering, over time, each component and interface exists in several versions. New versions originate from maintenance as well as from the incorporation of new requirements. The latter situation occurs especially if a component is used for more than one application, which is the normal situation for platform assets. The selection of component variants also has to take into account the version to be used. A later version is often better with regard to the quality of the component. Yet, it is possible that a new version of a component cannot be introduced into an application as it is not able to interact with other components. This may be due to changes in

[46] For a list of possible configuration mechanisms, see Section 12.5.

functionality, changes in quality support, and changes in interfaces, for example. In addition, it may be the case that the application has a combination of components that has not yet been thoroughly tested yet.

> ***Example 17-4:*** *Lock Control Configuration*
>
> The application with the sliding door and iris scan is built from the compiled components that make up the configuration. Figure 17-5 shows the configuration around the 'lock control' component. The grey components are application specific and the white ones are reusable domain components. The plug-in components know statically to which component they have to connect, whereas the components that provide the connection facility do not know the plug-in components statically. Since only statically known connections can be linked off-line, they are the only ones that are bound before loading the software on the target machine. Thus the 'authorise' interface of the 'electronic authentication lock plug-in' component is bound on the target hardware at initialisation time to the 'lock control' through the 'bind lock' interface. The 'bind lock' interface is statically known to the 'electronic authentication lock plug-in' and already bound during linking. During initialisation, the 'electronic authentication lock plug-in' component announces its 'authorise' interface to the 'lock control' component using the 'bind lock' interface. If component support, such as COM, .NET, or Java Beans, is available on the target machine, part of the linking can be done based on that. But also in that case, the statically known connections are bound first, and the dynamic ones next.

Responsibility for fixing errors

During integration tests and during maintenance in the field, problem reports are issued. Application engineering is responsible for resolving the reported problems. When the problems are related to application-specific components and interfaces, it is the responsibility of application realisation to fix them. However, when the problem is related to domain artefacts, the problem report has to be taken over by domain engineering for fixing.

17.5 Differences from Single-System Engineering

Working application

Application realisation provides a working application that is ready for testing. The application is based on the application architecture and reuses domain components and interfaces. This reduces realisation effort significantly. This means that:

- The application developer selects variants of the reusable domain components.
- The reusable domain component variants should be consistent with each other and conform to the application architecture.
- The application is built by configuring application-specific and reusable components and interfaces.

17.6 Summary

Realisation and configuration

Application realisation deals with designing in detail, implementing, and configuring components to produce an executable application. Interfaces are reused from the platform without changes, but components often have internal variation points, which have to be bound, e.g. by providing values for parameters. Newly developed application-specific components and specialised reusable components are configured and connected by their interfaces to assemble the application. After assembly, the application can be tested and deployed on the target hardware.

Klaus Pohl
Andreas Reuys

18

Application Testing

In this chapter you will learn:

- *About the interrelations of the application testing sub-process with the application requirements engineering, application design, application realisation, and the domain testing sub-processes.*
- *How to reuse domain test artefacts for a particular application.*
- *How traceability facilitates the reuse of domain test artefacts.*
- *A systematic way for deriving application test cases from domain test cases based on application requirements, application design, and application realisation artefacts.*
- *The principles of achieving sufficient test coverage in the system test.*

18.1 Introduction

Goal of application testing

The goal of the application testing sub-process is to achieve a sufficient quality of the application under test. Application testing thus complements the testing activities of domain testing.

Related sub-processes

The sub-processes and artefacts closely related to the application testing sub-process are highlighted in Fig. 18-1. Application testing reuses domain test artefacts. The unit test in application testing requires input from application realisation. The integration test requires input from application design, and the system test is performed on the basis of application requirements. The results of application testing are provided as feedback to the related sub-processes. Figure 18-2 shows the information flows between application testing and its related sub-processes.

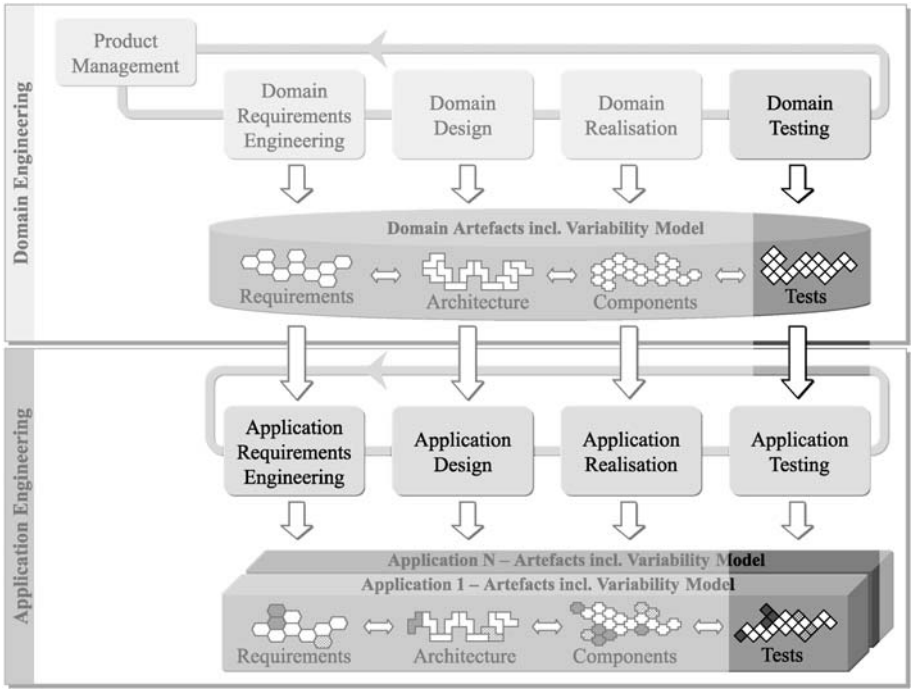


Fig. 18-1: Sub-processes and artefacts related to application testing

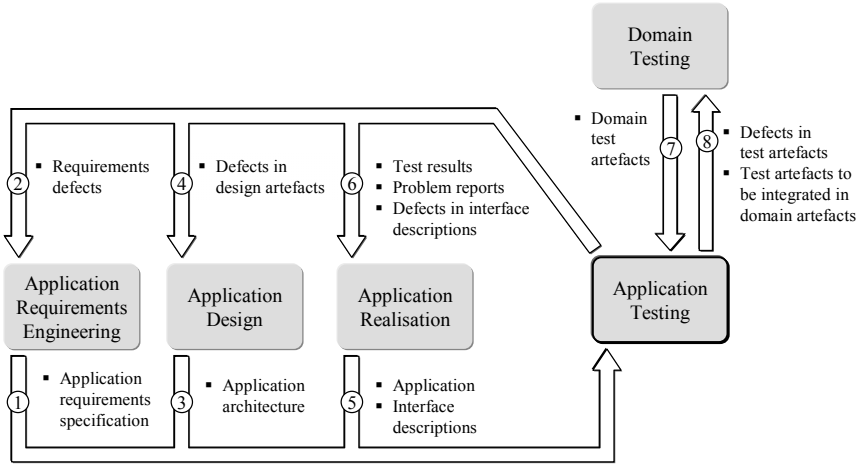


Fig. 18-2: Information flows between application testing and other sub-processes

18.1.1 Interrelation with Application Requirements Engineering

Application requirements

Application system testing validates the created application against the application requirements specification (① in Fig. 18-2). The system test that is performed in application testing employs the application requirements as test references. The test must ensure that the application properly realises the application requirements and that no requirement has been omitted. Furthermore, the application variability model defines the variability bindings for the application, and thus the variants that have been selected for the specific application. Application testing binds the variation points in the domain test artefacts according to the variability bindings in the application variability model.

Requirements defects

The creation of test cases is at the same time a validation of application requirements. Requirements defects such as ambiguous or incomplete requirements as well as errors in the configuration of variants are reported back to application requirements engineering (② in Fig. 18-2). Application requirements engineers must correct these defects before application testing can be completed.

18.1.2 Interrelation with Application Design

Application architecture

Application design determines the architecture of the application including its static structure and its dynamic behaviour. The application architecture (③ in Fig. 18-2) is used as input for application integration testing. Due to the binding of the variation points, all components of the application are available for testing. The components that are part of the application can be separated into components that are reused from the domain artefacts and

application-specific components, which encompass modified domain components as well as newly developed application-specific components. For reused domain components, reusable test artefacts are available, e.g. if the CRS (see Chapter 13) is applied.

Defects in design artefacts

During application integration testing, the application design is validated. Test engineers must ensure that the variability in the design has been bound properly and that the application design is testable. Whenever the test engineer cannot fully determine the integration test cases and the data required for a test case, an incompleteness or ambiguity has been detected. Any defects in design artefacts (④ in Fig. 18-2) detected in application testing are reported back to application design.

18.1.3 Interrelation with Application Realisation

Executable application

Application realisation builds the application (⑤ in Fig. 18-2) and provides the interface descriptions to be used as the test references for the unit test. The application unit test validates the components, which have been newly developed or modified by application realisation, but also repeats tests already performed in domain testing. The different test levels and the corresponding tests are described in more detail in Section 18.4. In addition, application testing performs tests related to the binding of the variability as explained in Section 18.3.

Test results

Application testing reports all test results, together with problem reports as well as the uncovered defects in interface descriptions, back to application realisation (④ in Fig. 18-2). The test results capture which test cases have been performed and whether the object under test passed or failed the test. The problem reports capture the observed deviations from the expected behaviour, which the object under test should possess according to the test reference. Defects in interface descriptions hamper test case design and must be corrected before testing can be completed.

18.1.4 Interrelation with Domain Testing

Domain test artefacts

In order to avoid developing tests from scratch for each application, domain testing provides application testing with reusable test artefacts (⑦ in Fig. 18-2). In order to perform the tests, application test engineers must bind the variation points in the domain test designs according to the application variability model.

Feedback to domain testing

Any defects in the domain test artefacts themselves are reported back to domain testing (first bullet of ⑧ in Fig. 18-2). In addition, application-specific test artefacts (second bullet of ⑧ in Fig. 18-2) created during application testing may be passed on to domain testing. Domain testing may integrate application-specific test artefacts into the domain artefacts, for

example if the application-specific test artefacts are relevant for more than one application.

18.2 Domain Test Artefact Reuse

Implications of the CRS

The key idea of establishing test artefact reuse is to develop test artefacts once for the entire product line, include them in the domain artefacts, and reuse them for multiple applications. To achieve a sufficient degree of reuse, domain test artefacts must be reused for common as well as for variable parts of the application. Following the CRS (Definition 13-10), domain testing prepares test cases with explicit common and variable parts. Test cases for common parts can be reused as they are. For test cases that contain variability the variation points must be bound according to the application variability model before the test cases can be executed.

Implications of the SAS

If the SAS (Definition 13-9) is applied, the test cases developed for the sample application can be used as a basis for developing test cases for the application under test. It is quite likely that the test cases of the sample application have to be adapted. Therefore, the variants selected for the sample application are compared with the variants selected for the application under test. Based on the differences in the variability bindings, the required adaptations of the test artefacts can be determined. However, in case of large differences in the applications, significant rework might be required to create the application test artefacts. Note that additional test cases must be defined to validate application-specific requirements.

Focus on system test

In the following, we elaborate on two essential prerequisites for establishing an efficient reuse of test artefacts: dealing with variability and the use of traceability links. We thereby focus on system tests. However, the basic principles presented also hold for integration and unit tests.

18.2.1 Dealing with Variability

Binding variability

The application test engineer has to understand the variation points and variants defined in the domain artefacts and know how to bind the variability defined in the domain test artefacts. Application test artefacts are built based on domain test artefacts. The test plans, test cases, and test case scenarios have to be adapted for the specific application. The application test engineer thus has to learn how to bind the variability defined in the domain test artefacts according to the application variability model. Example 18-1 illustrates the binding of variability in a domain test case scenario.

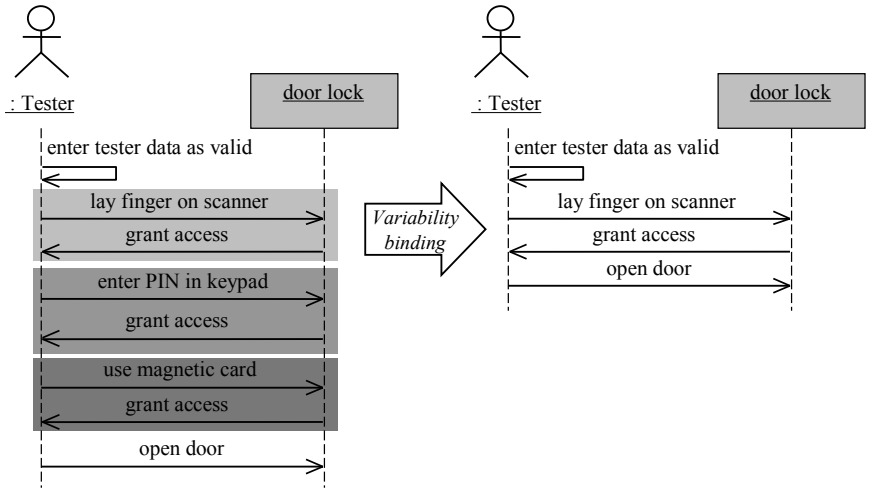


Fig. 18-3: Example of variability binding in test case scenarios

Example 18-1: *Dealing with Variability in Application Testing*

The application test engineer of a home automation application considers the domain test case scenario depicted on the left side of Fig. 18-3 (this scenario is explained in more detail in Section 8.3). The domain test case scenario contains three variants: 'fingerprint scanner', 'keypad', and 'magnetic card reader'. According to the application variability model (not depicted in Fig. 18-3), the variant 'fingerprint scanner' has been bound for the application under test. In order to derive the application test case scenario, the test engineer binds this variant also in the domain test case scenario. The result is depicted on the right side of Fig. 18-3.

18.2.2 Use of Traceability Links

Retrieval of test artefacts

To allow for an efficient reuse of test artefacts it is necessary to support the retrieval of the applicable domain test cases. Figure 18-4 presents the basic idea. The trace information captured by application requirements engineering (Chapter 15) is used to detect the reused domain requirements artefacts. From domain testing, it is clear which test artefact relates to which requirement. Thus, based on the domain requirements, the reusable domain test artefacts can be retrieved. The domain test artefacts retrieved can be used to test the application requirements which correspond to the domain requirements related to the domain test artefacts.

Requirements deltas

Traceability is the first keystone for structured reuse. The second keystone is the requirements delta information that is contained in the application requirements specification. The deltas are the basis for determining whether a test case can be reused without changes, must be adapted, or created anew.

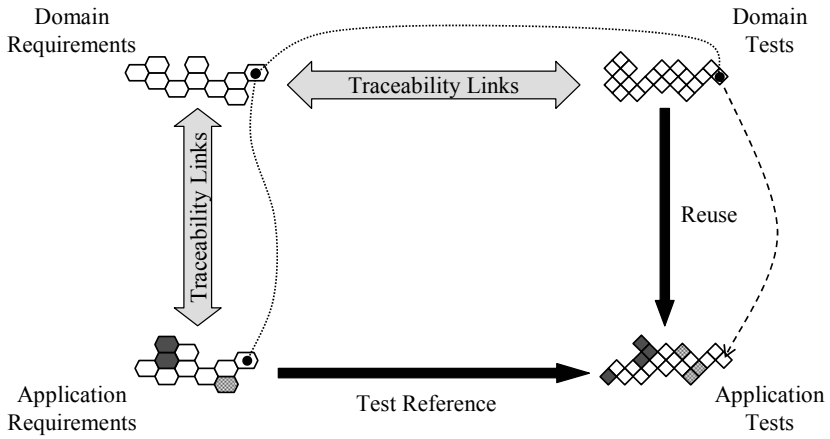


Fig. 18-4: Traceability between requirements and system test artefacts

Recording and using traces

A detailed process description for application testing defines when to record and when to use trace information. Although we do not deal with trace information at this level of detail, clear rules for the recording and usage of trace information are required in industrial software product line engineering to facilitate the reuse of domain test artefacts.

Example 18-2: *Retrieving Domain Artefacts*

In Example 18-1, a trace exists between the requirements artefacts for electronic door locks and the test case scenario of Fig. 18-3. The dependency between the two artefacts is recorded by a traceability link, which we refer to as "DomainDoorLockTestTrace". Whenever one of the three door lock variants is incorporated into an application, the dependency between the domain and the corresponding application requirements artefact is recorded as well.

The "DomainDoorLockTestTrace" is used to identify the reusable domain test case. The domain test case is reused for a new application test case that validates the realisation of the corresponding application requirement.

18.3 Tests Related to Variability

Defect categories

The task of application testing is to validate that the application under test complies with the test references from domain engineering. This implies that the binding of variability and the configuration realised in the application are checked for correctness. More precisely, the application test engineer has to check if:

- Application engineers have bound variants for the application that should not be part of the application.
- Application engineers have omitted variants that should be bound for the application.
- Application engineers have configured the application in a way that violates the constraint and/or variability dependencies.

If the application contains variants that should not be part of it, different kinds of errors may occur. The usability of the application may be affected as the customer finds the variant impracticable or the variant consumes additional resources and thus affects system performance. If a variant that should bound for the application is omitted, the functionality and/or quality related to this variant is missing in the application. Furthermore, the violation of dependencies may lead to a malfunctioning application. For example, if a variant that is required by another variant or variation point is not included in the application, the components that require this variant may not work properly. If an "excludes" dependency is violated and thus a variant that is in conflict with another variant or variation point is included in the application, the conflicts between the different variants may lead to errors.

Tests

Specific tests are required to detect defects in the binding of variability and the configuration of an application since the proper functioning of an application may be affected by such defects. On the other hand, if a dependency that has strategical reasons is violated, this defect may be difficult to detect. Only a specific test can reveal this type of deviation from the application requirements specification. In the following, we thus introduce two types of tests for detecting defects in the variability bindings and the configuration of an application.

> ***Definition 18-1:*** *Variant Absence Test*
>
> The variant absence test ensures that an application does not include variants that were not defined to be included in the application.

Variant absence test

The variant absence test verifies that no more than the selected variants are incorporated into the application. To check whether a variant is present or

not, the variant absence test may call a function that is provided by the variant and observe the reaction of the application. The variant absence test affects all test levels:

- The unit variant absence test verifies the absence of variants that influence code within a component. This kind of variability is often realised by compile time configuration mechanisms (e.g. IFDEF statements).
- The integration variant absence test verifies the absence of variants that influence entire components. This kind of variability is often realised by link time, load time, or run-time configuration mechanisms (e.g. registry).
- The system variant absence test verifies the absence of variants that have influence on major system features and may therefore be distributed over multiple components. This kind of variability is also realised by link time, load time, or run-time configuration mechanisms.

Definition 18-2: *Application Dependency Test*

The application dependency test checks if the application is in conformance with the constraint and variability dependencies specified in the domain and application variability models.

Conformance to variability models

The application dependency test has to detect configurations of variants that are not allowed according to the domain and application variability models. This task can be subdivided into two sub-tasks:

- Checking whether the presence of variants violates any restrictions imposed by variability and constraint dependencies (e.g. “excludes” constraint dependencies).
- Checking whether the absence of variants violates any restrictions (e.g. “mandatory” variability dependencies or the “[min..max]” range of an alternative choice).

Example 18-3: *Application Dependency Test for Heating Control*

The automatic heating control requires automatic windows to prevent the waste of energy. Once the home automation system detects that a room is too cold, the windows are closed and heating is turned on. The home automation system for a specific customer includes automatic heating control. Hence, the application dependency test checks whether the application also contains the automatic windows variant.

Similar to the variant absence test, the application dependency test affects all test levels. At each level those dependencies are tested that affect realisation artefacts at the corresponding level of granularity, i.e. code excerpts, components, or the entire application.

18.4 Testing Variability at Different Test Levels

In this section, we consider the influence of variability on the test levels in application testing.

18.4.1 Application Unit Test

Retesting due to variability

The goal of the application unit test is to validate single components against the component specifications. The application test engineers reuse the domain unit test cases for the common functionality of the unit. It is necessary to repeat unit tests even for common functionality as the specific configuration of the application may influence the test results (Section 13.3). The common parts of a component may not work properly due to a particular combination of variants or due to application-specific modifications.

> ***Example 18-4:*** *Necessity to Reapply Domain Unit Test Cases*
>
> The door sensor plug-in, which is a component of the door lock control subsystem, supports control devices to which up to eight door sensors can be connected. During domain testing the plug-in was tested with different configurations of sensors, and passed the test. The domain unit test cases are reexecuted during application testing, as it is still unclear whether the plug-in component behaves correctly for the particular configuration of sensors required for the application.

18.4.2 Application Integration Test

Test of component interactions

The goal of the application integration test is to validate the interactions between the components of the application. This includes the validation of the interactions between common components, bound variants of variable components, as well as application-specific components. During the application integration test, test cases that have already been performed in domain testing may have to be reexecuted to validate the interactions of all components in the specific context of the application (Section 13.3). The inputs for the application integration test are components that have passed the application unit test.

Example 18-5: *Application Integration of the Home Security System*

The domain integration test case for electronic door locks contains variability as there are three different variants (fingerprint scanner, keypad, and magnetic card reader). The application under test includes a door lock management component and three door control devices, each with a magnetic card reader and a lock actuator. The application integration test cases are created by reusing the domain integration test cases and by binding the variants defined in the application variability model. The test cases are used to validate the interactions between the three magnetic card locks and the server.

18.4.3 Application System Test

Test of application features

The goal of the application system test is to validate that the application satisfies the application requirements specification. Ideally, all requirements are tested to obtain a detailed assessment of the quality of the application. Most of the system test cases have to be performed during application system testing due to the absence of application-specific variants in domain engineering and the configurability of domain artefacts (see Chapter 13). If the commonality and reuse strategy is applied, application test engineers can reuse predefined domain system test artefacts by binding the variants defined in the application variability model.

Example 18-6: *System Test of a Home Automation Application*

A system test case for the home security part of the home automation application is the activation of the vacation mode. The test case scenario consists of the following steps. The user authenticates against the system, activates indoor and outdoor surveillance, and locks the doors. To create the application system test case, the test engineer reuses a domain system test case. As the domain system case includes different authentication mechanisms, different surveillance devices, etc., the test engineer binds the authentication variants, the surveillance variants, etc., as defined in the application variability model.

18.5 Application Test Coverage

Requirements coverage by category

In this subsection, we describe the different types of tests that have to be performed in application system testing. The different types are based on a classification of the application requirements:

- *Reused common requirements artefacts*: This category includes all application requirements that are reused common domain requirements.

- *Reused variable requirements*: This category includes all application requirements that are reused domain requirements with bound variability.
- *Adapted requirements*: This category includes all application requirements that are reused domain requirements containing application-specific adaptations.
- *New requirements*: This category includes all application requirements that are purely application-specific and are not derived from domain requirements.

18.5.1 Application Commonality Test

Retesting common requirements

The application commonality test covers the common parts of requirements models. It ensures that the common requirements of the product line are tested in the context of an application. Test cases for common requirements are developed and executed in domain testing. If, for instance, a common component is placed in the context of a specific application with its specific selection of variants, the component can fail even though it has not changed. Coupling mechanisms like shared variables or inheritance can cause such failures, even if the components have been tested extensively. Consequently, domain test cases for common requirements are reexecuted in application testing.

> ***Example 18-7:*** *Testing Common Requirements*
>
> One part of every application is the motion detection sensor. Test cases for the motion detection sensor are created during domain testing and executed on the sample application. The test case is reexecuted for each application to ensure correct behaviour.

18.5.2 Application Variant Test

Testing variants

The application variant test verifies that all selected variants are part of the application and ensures the correct behaviour of these variants. The test artefacts for the application variant test can be reused from domain test artefacts. Application test engineers bind the variants based on the application variability model.

> ***Example 18-8:*** *Testing Variants*
>
> The application under test contains exactly three magnetic door locks. The application variant test must ensure that there are three magnetic door locks and that each of them locks the specified door.

18.5.3 Application-specific Tests

The application-specific tests deal with testing new application requirements and adapted domain requirements.

Testing adapted requirements

The application modification test covers adapted requirements, i.e. the parts of application requirements models that are modified versions of domain requirements models. For this category of requirements it is not possible to reuse the domain test artefacts without adapting them.

> ***Example 18-9:*** *Testing Adapted Requirements*
>
> Another set of magnetic card locks from another provider has to be incorporated into an application. The test cases regarding the magnetic card locks must be adapted to cope with the new hardware, for instance, pull the card through a card reader instead of placing it on the card reader.

Testing new requirements

The application extension test covers parts of the application requirements models that were newly developed for a specific application. The test artefacts for this category of requirements have to be created from scratch. In addition, the newly implemented artefacts typically have a significantly higher defect density than reused artefacts.

> ***Example 18-10:*** *Testing New Application Requirements*
>
> An iris scanner lock is required for an application. Therefore, the application extension test must ensure that the requirements are realised within the application. New test cases have to be derived to validate this requirement.

18.6 Application Test Activities

The application test process description provides guidance for the entire application test process. It includes instructions on how to derive application test artefacts from domain test artefacts. The application test process consists of three main activities: application test planning, application test specification, and application test execution.

18.6.1 Application Test Planning

Adaptation of the test plan

During application test planning the test engineers create an application test plan for the product line application. The test plan differs from application to application. For example, there may be some variants that have already been tested, or there may be new application-specific requirements, which require

the definition of a new test plan. Consequently, the required effort for application testing also differs from application to application. The estimation of test effort can be based on a common scheme defined by domain testing. The test strategy and the tool support for executing the tests have already been determined during domain engineering. Only in exceptional cases are they adapted during application testing (e.g. in pilot projects for evaluating a new tool or a modified strategy). Such a case may be, for instance, an attempt to improve the efficiency of domain test artefact reuse.

> ***Example 18-11:*** *Application Test Planning for the Home Automation System*
>
> The differences between application and domain requirements artefacts and the previously performed tests are considered. If three magnetic door locks have to be built into the home, the interaction between three door locks and the server has to be tested in the application integration test. This validates that the application works with the three locks.
>
> These test cases are identified on all test levels and for all types of application tests (e.g. variant absence tests). They are defined as part of the test plan. Resources are allocated to the test cases, e.g. one test engineer must perform the door lock interaction tests within two days.

18.6.2 Application Test Specification

Logical and detailed test cases

During application test specification, test engineers create logical test cases, detailed test cases, as well as the respective test case scenarios for the application. The effort for the specification activity depends on the achievable degree of domain test artefact reuse:

- For application commonality tests, logical and detailed test cases are available from domain testing and can be reused in application testing.
- For application variant tests and, to some degree, for application-specific tests (Section 8.3), logical test cases can be reused from domain test artefacts (Section 13.7). Detailed test cases have to be developed by the application test engineers.
- Application-specific tests must be developed from scratch (including the logical and detailed test cases) or obtained by adapting domain test artefacts.

Example 18-12: *Application Test Specification for Electronic Door Locks*

During the application test specification activity, test engineers create the detailed test cases for electronic door locks and adapt the test case scenarios for the selected authentication mechanism. As the application uses only magnetic sensors, the other two variants (keypad and fingerprint sensors) are omitted. Furthermore, the exact number of locks is incorporated into the detailed application test cases and application test case scenarios.

18.6.3 Application Test Execution

Perform test cases and record results

During application test execution, the application test engineers perform the specified test cases on the application. They record the results and complete the tests by determining the error classes. This also includes reporting the detected defects to the other sub-processes (see Section 18.1).

Example 18-13: *Application Test Execution for the Electronic Door Locks*

The test cases and test case scenarios for validating the three magnetic door locks and the interplay with the authentication server are executed. Defects in the 'electronic door lock control' component, in the interaction with the 'authentication' component, and in the entire application (e.g. failure during simultaneous access) are uncovered and reported to the developers.

18.7 Differences from Single-System Engineering

Domain and application artefacts

As for single-system testing, the goal of application testing in software product line engineering is to ensure a sufficient quality of an application by performing a set of tests that satisfies the chosen coverage criterion. In contrast to single-system engineering, the test engineers have to consider that the requirements as well as the application to be tested are created partly during domain engineering and partly during application engineering. Application requirements and components that are identical to domain artefacts are tested by repeating tests that have been created in domain testing. Application-specific artefacts are tested in a similar way as in single-system engineering since the test cases have to be newly developed or adapted.

Tests related to variability

Application test engineers must validate that the variability binding for the application complies with the application requirements specification, or, more precisely, with the application variability model. In addition, the appli-

cation must not violate any restrictions imposed by the domain variability model. In single-system engineering, there is usually no need to perform this kind of test.

Reuse

In the application testing sub-process in software product line engineering, the test process description as well as a large part of the required test artefacts do not have to be developed from scratch as they are both available from the domain artefacts, partly in a generic form. After binding the appropriate variants and adapting or concretising the artefacts, they can be employed in application testing (see Chapter 8 for more details on variability in test artefacts).

18.8 Summary

Extensive application test

During application testing a product line application is validated against its specification. For this purpose, a set of application test cases is defined that includes all test levels and fulfils a suitable coverage criterion. A thorough test, even of reused code, is necessary due to dependencies between reused and application-specific parts.

Efficient reuse

Many application test artefacts are reused domain test artefacts. To establish an efficient reuse process, for instance, application test engineers must be trained to deal with variable test artefacts. In addition, traceability links must be recorded in domain engineering as well as in application engineering activities that enable test engineers to locate easily the test artefacts for reuse.

Application test artefacts

The test cases performed together with the recorded test results indicate the level of quality achieved in an application. To ensure a traceable and repeatable application test process, the test documentation is recorded as an application artefact and interrelated with the other application artefacts, even if part of it is derived from domain artefacts. Just like other application-specific artefacts, test artefacts developed for a specific application might be included in the domain artefacts if they are of interest for other applications.